Threat Analytics Across Payment Products: Evidence Requirements for Agent Investigations

Jonathan Matkowsky

New York, New York. Correspondence: Jonathan@matkowsky.com

10 November 2026 · Accepted working draft for version 1.1 · Not published · Not peer reviewed

Abstract

Abuse of a payment platform can cross products and accounts, and the platform's own protective decisions change which outcomes can later be observed. A blocked payment cannot produce a chargeback; an incomplete search cannot establish absence; a reused conclusion cannot corroborate itself. This paper specifies the records an AI agent needs in order to investigate such cases under human authority. The specification rests on three requirements. First, each outcome record links to the control decision that may have shaped it, with separate fields for reporting delay, outcomes the decision made unobservable, and false reports. Second, a gateway outside the agent records what each query actually covered, and every fact retains both the time it happened and the time the system learned it. Third, a person must release a conclusion before it enters organizational memory for reuse by later investigations; a released conclusion serves only as a hypothesis or context, and any record that an agent output informed must link back to that output.

The specification connects these requirements to 71 signals, the kinds of records an investigator can check, and 24 responsibilities for investigation, review and reuse. Public sources supply the design premises, and Stripe's public product pages and documentation provide an illustrative product scope. Two hypothetical investigations show how missing context changes the conclusions that the same observations can support, and an evaluation plan proposes tests of evidential quality, operational cost and agent benefit. The contribution is a joint specification for investigation across payment products, built on established methods and recent agent-governance designs. It is a synthesis, not a new method or a measured result: the specification has not been built or evaluated, and no practitioner has reviewed it.

Keywords: payment fraud, abuse investigation, threat intelligence, selective labels, agent memory, provenance, human oversight

1. Introduction

Consider an investigator examining several storefronts that share payment accounts. A low chargeback count could reflect little abuse, payments blocked before completion, refunds that prevented disputes, or reports recorded under another account. Finding no earlier suspicious requests could reflect missing history rather than clean history. An earlier intelligence report could suggest where to look, yet add no independent evidence. Each possibility calls for a different conclusion and a different next step.

The difficulty is not simply connecting more data. The connections must preserve what each record can establish. A payment log records activity. A risk score records a model's assessment. A control decision records what the platform allowed, blocked or restricted. An outcome label records a reported result, which may arrive late or be wrong. When these distinctions disappear, an investigation can mistake a decision for proof, an unobservable outcome for a benign one, or repeated analysis for corroboration.

Each of these failures has its own research literature. Selective-labels and counterfactual-evaluation methods address outcomes hidden by decisions. Temporal and incomplete-database research addresses when a fact became known and whether a query answer is complete. Provenance standards and agent-memory designs address the status of derived conclusions. These bodies of work mostly treat the failures one at a time. Some recent agent-memory and decision-record designs address two or three of them in part, and they differ on who may authorize reuse of a conclusion and what a reused conclusion can establish (Section 2). An investigation that spans payment products can meet all three failures in one case, so its records must keep all three visible together.

This paper specifies records for threat analytics: investigation that connects product activity to intelligence, proposed detections and control recommendations, then learns from later outcomes. The setting is a payment platform offering several products to businesses and their end customers. The investigator of interest is an AI agent: software that uses a language model to choose queries and interpret their results. It works within human-assigned tasks and enforced access limits. It can investigate and recommend; authorized people or accountable operational systems decide what happens to accounts, payments and controls.

The design addresses three research questions:

RQ1: What records and analytical capabilities support investigation across payment products while preserving the meaning of observations and outcomes?

RQ2: How can an agent test explanations and prepare useful outputs while people retain review, release and operational authority?

RQ3: How can later outcomes and reusable lessons inform investigation without concealing the effects of controls or recycling conclusions as evidence?

The contribution is three requirements on the investigation's records:

C1 links outcomes to decisions. An outcome channel is a route, such as a chargeback, by which an outcome can be reported. Each outcome record links to the control decision that may have shaped it, and to the status of each channel that the decision closed, delayed or changed. Late reports, outcomes that a control made unobservable, and false reports occupy separate fields. An evaluation can then state the joint assumptions it needs about delay, observability and report truth, each of which may depend on the decision (Section 7).

C2 records query coverage and learned time. Query coverage is what a query actually covered: the data and period it examined, and whether its result was complete. Learned time is when a fact became available to the system's queries. Every query passes through a gateway, a checkpoint outside the agent, which records its coverage. Every fact keeps both when it happened and when the system learned it (Sections 5.1, 5.3 and 6.2).

C3 requires human release before promotion into organizational memory. No agent output enters the evidence plane, which stores observations, as evidence of the activity under investigation (Sections 4.2 and 5.3); the agent's own outputs are recorded only as observations of what the agent did (Section 6.5). A person releases a conclusion into organizational memory, where it serves only as a hypothesis or context. Any record that an agent output informed links back to that output and counts as dependent, not as independent corroboration (Sections 4.2, 6.4 and 7.3).

The requirements work together. C1 identifies which outcomes the platform's decisions left observable. C2 establishes what information an investigation actually examined and could have known. C3 prevents the resulting interpretation from returning as independent support for itself. A record can satisfy one requirement and still mislead an investigation through a failure of another.

Two artifacts make the specification inspectable: an inventory of 71 signals and an architecture of 24 responsibilities. Sixty-eight signals map to abuse mechanisms, meaning ways abuse can occur; three apply across mechanisms. The responsibilities cover investigation, review, release, tested reuse of lessons and security of the investigation system. Two hypothetical investigations then show which conclusions survive when particular records are missing.

The intended readers design, audit or evaluate investigation systems. Stripe's public product pages and documentation supply an illustrative product scope, not a description of its internal systems. The paper synthesizes public evidence and proposes a design; it reports no implementation or measurement. Section 2 establishes its relationship to prior work and Section 3 explains how it was derived. Sections 4 and 5 answer RQ1, with Table 4 in Section 6.3 adding the agent's analytical capabilities; Section 6 answers RQ2 and Section 7 answers RQ3. Section 8 shows the consequences of the design in two hypothetical investigations, and Section 9 states how its value would be tested.

2. Relationship to prior work

The three requirements combine established ideas in a specific setting. This section identifies the closest precedents for each requirement and the choices the specification adds. Supplement S9 gives the extended comparison, with the nearest precedents for each requirement summarized in Table S9.1; S1 provides source passages, publication status and a 50-source requirements matrix.

2.1 Decisions shape the outcomes used to judge them

When a decision determines which cases receive outcome labels, evaluation faces the selective labels problem [1, 2]. Counterfactual analysis of logged decisions is most direct when the decisions were randomized with known probabilities. Each logged sample is then reweighted to reflect its probability under another decision rule, so that average outcomes under that rule can be estimated [3]. All of these methods require assumptions about which comparisons the data support [4]. Consumer credit met the same problem earlier as reject inference: a lender observes repayment only for the applicants it financed and asks whether, and under what assumptions, the outcomes of rejected applicants can be inferred. A review of common reject-inference methods finds their assumptions mostly implicit. Without financing applicants who would otherwise be rejected, the methods cannot be compared by standard model selection, and no method is uniformly better [5].

Payment research makes the reporting problem concrete. TitAnt trains on delayed user fraud reports [6]. Dal Pozzolo and colleagues distinguish prompt investigator feedback from later labels and show how alert-driven review biases training data [7]. An e-commerce research agenda likewise separates suspicion on blocked transactions from outcome labels [8]. A theoretical payments preprint models authorization, whether fraud is reported, whether the report has arrived by training time, and label corruption, for one reporting channel [9]. Related preprints examine delayed, incomplete or incorrect feedback [10, 11]. Chapelle's advertising model supplies a precedent for treating a click with no conversion yet as unlabeled rather than negative [12]. Payment practitioners have described estimating outcomes under decisions that were not taken. A 2015 talk abstract describes adding a small amount of randomness to production fraud scoring so that a blocking model's precision and recall can be estimated from production data [13]. Adyen evaluates a new policy for suggesting payment-time interventions, aimed at authorization rates rather than fraud, from transactions logged under the existing policy [14]. Two recent design preprints come closest to C1's record. One, the Decision Event Schema, seals each decision record once its hash is computed. Facts learned later, such as the arrival of ground truth, are appended as linked records that reference the decision,

and links between decisions carry a type [15]. The other, an execution-coverage model, links records to decisions and distinguishes refused execution branches from late reports [16].

C1 applies these ideas to several outcome channels at once. It records what each decision closed, delayed or changed, separately for each channel and read time. A blocked card payment cannot produce a chargeback on that attempt, but an incident finding remains possible. C1 keeps the decision as the unit of record, as the Decision Event Schema does. Each outcome report is a separate appended record. It links to every decision that may have shaped it, and each link states that decision's role. The schema has no field for outcome channels, but C1's fields could extend it rather than replace it (Section 5.2). The proposed records expose the assumptions needed to interpret such outcomes; they do not supply a validated estimator. Section 7 develops this distinction.

2.2 Evidence depends on coverage, time and origin

Temporal databases distinguish when a fact held from when it was stored [17], and agent-memory designs retain the same two kinds of time [18–20]. The execution-coverage model of Section 2.1 also separates event time from verifier receipt time, and it judges whether a verifier's evidence was complete at a stated cutoff from the records then due [16]. Research on incomplete databases relates the completeness of a query answer to which data are known to be complete [21]. These precedents matter whenever a query returns no matching records or a relationship becomes known after its effective date. Quipu gates writes to a knowledge graph while retaining both kinds of time [22]. PROVSEEK's agent cites audit-event identifiers for its claims and reports artifacts it searched for and did not find, but that report is the agent's own account of its search [23].

C2 combines explicit query coverage with learned time. The execution-coverage model asks whether the records an execution owes had arrived by a cutoff. It also separates a record missing from one retrieval from a record missing from a verifier view whose closure is justified. The basis for that closure includes retrieval coverage and retention, and the model specifies it without designing its implementation. C2 records what each investigative query examined. A gateway outside the agent records the requested and examined data, access limits and known gaps. Each fact's learned time is when it became available to queries. A relationship learned today cannot appear in yesterday's reconstructed decision merely because it was in effect then, although later outcomes may still evaluate that fixed decision under a stated outcome window.

The W3C PROV data model expresses the derivation of one entity from another [24], and decision provenance records the data flows surrounding decisions [25]. STIX separates observed data from intelligence assertions and opinions [26], and US intelligence analytic standards ask analytic products to distinguish underlying information from assumptions and judgments [27]. For agent investigations, those distinctions must survive retrieval and reuse. The agent's own account of its work cannot establish what its tools actually examined; independently written run records are the relevant precedent [28].

2.3 Released conclusions remain conclusions

Truth-maintenance research records the support for conclusions and revises them when that support changes [29, 30]. Belief-revision work on threat intelligence asks that intelligence state its source and its chain of modifications [31]. A conceptual framework for language agents separates working, episodic, semantic and procedural memory [32], and a survey argues that agent memory should keep raw observations, extracted facts and inferred items apart, with derivation links [33]. A recent

agent-memory design keeps an immutable raw log beneath its summaries and links each summary to the raw pages that support it [34]. Together these establish substantial precedent for C3.

The closest designs differ in who authorizes reuse and what a reused item can establish. Anamnesis starts model-derived claims as candidates, and no memory operation can raise their authority without a separately recorded grant; the grantor need not be a human [20]. GovMem groups correlated agent traces by their dependencies when assessing support and permits automatic promotion or human review [35]. Quipu's write gate requires human approval only under some policies [22]. A data-governance preprint is closer on authorization: no agent-proposed metadata decision is committed before a steward reviews it, and a proposed gate admits only verified records as precedent, records each promotion and lets a reviewer demote a record, flagging the decisions that relied on it. It does not say who authorizes promotion or what a retrieved precedent may establish [36]. AgenticCyOps provides analyst oversight and an audited memory-write path, without specifying human release for every reused item [37]. A threat-intelligence architecture requires human approval before operational action or dissemination, but does not state that analyses kept in its institutional-memory store pass that gate [38]. In the illustrative provider's agent-assisted fraud modeling, engineers review the agents' hypotheses and results and decide what to deploy, and for one fraud attack the agents were given results from earlier experiments and investigations; the account does not describe a release step for those earlier results [39]. Vendor architectures draw related lines. One keeps an append-only event record, a maintained fact record and a decision record that binds selected evidence and facts to policy, approval, action and outcome, with human or automated approval [40]. Another states that agent-generated summaries are not treated as authoritative business records [41]. Neither defines outcome channels, query coverage or a release rule for reusing conclusions. Agile-V binds evidence to the exact artifact and links effects to authorization records; its governing profile can require human authority [42]. A formal study distinguishes independent observations from reports derived from shared evidence and requires a dependence model as well as ancestry [43]. A threat-detection design comes closest on reuse as evidence. It tags each detection compiled from one of its hypotheses and drops tagged observations at ingest, so the system's own conclusions cannot re-enter as fresh evidence. It leaves as a residual risk an analyst who writes such an alert into a report that is later ingested, and its reference implementation does not yet exercise the rule [44].

C3 adopts human release for every conclusion entering organizational memory. Unlike a verified precedent in the data-governance design, a released conclusion is reused only as context or a hypothesis; release does not turn it into an observation. Records informed by an agent output retain their dependence on it, including labels, detections and control decisions. Because the interfaces that write these records add the links, they also cover a person who writes a record while an agent output is open in the case, part of the relay the threat-detection design leaves open; a missing link still does not establish independence (Section 4.2). This is a design choice whose benefit and cost require testing, particularly through the agent-benefit and release-gate evaluations (E5 and E8, Section 9.1); it is not a claim that human approval guarantees truth.

2.4 What the combined specification contributes

Industrial payment systems supply examples of fast decisions and relationships between accounts [6, 45]. Payment providers' public documentation already exposes parts of the first two requirements. The illustrative provider lets businesses query which rules affected each charge and join disputes to charges [46]. Its dispute analytics warns that rates by payment date can look lower for recent payments because customers can wait weeks to dispute [47], and each scheduled query run records

the as-of time of the data snapshot it queried [48]. Another provider's risk dashboard shows whether transactions its risk engine allowed later resulted in a chargeback, and lists separately those it blocked [49]. A payment-processing patent application describes adding a treatment result, such as the outcome of an authentication step, to the feedback on a transaction, labeling transactions canceled after the treatment as canceled, and notes that fraud can be reported up to 100 days later [50]. The specification states what such joins, labels and timestamps must preserve for an investigation: which outcomes each decision closed, delayed or changed, and which reports are late or false. A threat-intelligence reference model supplies analysis, quality-control and feedback functions for intelligence products, and traces who created and modified each product [51]. A governance synthesis for fraud risk decision systems combines decision traces, measures of evidence sufficiency under delayed labels and monitoring without labels, and extends them analytically to multi-agent systems [52]. It rates architectures by how many decision-record properties they fill by construction; it specifies no outcome channels, query coverage record or release rule for reuse. Table 1 locates these capabilities across research families.

Table 1. Stages described in the cited research. E means explicitly described; P means partial coverage or a different scope; a dot means not established by the cited material. Each family receives the strongest source-level code in Supplement S1, so one row may combine several systems. L1: signal sources; L2: ingestion and stores; L3: entities, correlation and knowledge; L4: investigation; L5: outputs; L6: feedback; L7: governance; L8: learning and correction.

Research family and inspected examples	L1	L2	L3	L4	L5	L6	L7	L8	Payment domain
Industrial payment and risk systems: TitAnt, GEM	P	E	P	.	P	P	.	P	yes
Illustrative platform's (Stripe's) engineering publications, 2019 to 2025	P	E	P	P	P	E	.	P	yes
Threat-intelligence and security-operations architectures: reference models, intelligence lifecycle and agent benchmarks	P	P	E	E	E	P	E	E	no
Preprints on agents making fraud decisions: SR-Fraud and Sharma's auditable agent	P	P	P	E	P	E	P	E	yes

Two payment-agent preprints show why specification and operational benefit must be kept apart. SR-Fraud separates scoring from reflection and tests proposed rules before reuse. On one replay benchmark it reports better detection than its scoring agent alone. With one of the three language models it reports, its point estimates also exceed those of static and periodically retrained CatBoost classifiers; with the other two, some fall below the periodically retrained classifier. For that model, against the periodically retrained classifier at that classifier's 5 percent operating point, its recall and dollar-weighted recall gains are significant, but when both block the same number of transactions none of the differences is significant. Its point-estimate gains grow with users' prior history. On the 64 percent of transactions with any prior history, its recall, F1 and dollar-weighted recall gains are significant at that classifier's 5 percent operating point, where SR-Fraud blocks 6.08 percent of all transactions; matched results are not reported by history. At the matched block count, its recall in the week-8 fraud burst is about 24 percentage points higher, without an interval [53]. On 60 balanced synthetic cases that its classifier scored between 0.2 and 0.8, Sharma's agent was correct in 65.0 percent, below the 71.7 percent from thresholding the same classifier score at 0.5. Six of its eight overrides turned a correct decision into an error [54]. Neither result establishes general agent performance: one rests on a single non-public replay benchmark, and the other on a small,

balanced synthetic sample. Together they motivate comparing an agent with simpler alternatives given the same information.

This paper connects decision-shaped outcomes, query coverage and controlled reuse in one payment-investigation specification. The 71 signals identify candidate records, the 24 responsibilities assign investigation and review functions, and the interfaces preserve the meaning of what passes between them. The traces show which inferences lose support when that context is removed; they establish logical consequences within hypothetical cases, not improved detection. The focused comparison does not establish uniqueness or exhaustive coverage, and operational value depends on the evaluations in Section 9.

3. Deriving the specification from public evidence

The design starts from documented product capabilities and asks how an adversary could misuse each one. For each abuse mechanism it then identifies the records needed to distinguish the malicious explanation from plausible alternatives. Stripe's public product catalog, roadmap and documentation define the illustrative scope; research, engineering accounts, incident reports and guidance supply the design premises.

The scope, read on 27 September 2026, contains 88 product and feature entries. A retained mechanism must target a documented capability, serve a plausible adversary goal and have supporting public evidence. Public job postings from the illustrative provider, Stripe, for abuse, risk and machine-learning roles helped prioritize topics as a statement of its current priorities, not as evidence of incidents [55, 56]. Supplement S2 records the 99 candidate mechanisms, how postings set priorities, the nine search paths and each candidate's final disposition; every mapped mechanism also has supporting sources beyond postings.

The resulting inventory contains 71 signals in 12 families. Sixty-eight connect to 61 mechanisms through 163 links, and three apply across mechanisms. Nine links are explicitly inferential; the others use sourced premises to motivate record design without validating a detector. Each derived feature, a value computed from other records, identifies its inputs. Supplement S2 gives the disposition of the other 38 candidates.

Three published schemas describe the platform's application programming interface (API), the interface through which software requests operations and receives data. Fixed at commit `18fa2cc7 68024b47789dc6fa243acd48478acac2`, they provide a reproducible check for partial public counterparts of the proposed records [57]. Supplement S5 supplies the object, field or event supporting each match.

Supplement S1 records the version, relevant passages and use of each of the 122 cited works, and compares 50 sources against the three requirements. Supplement S3 lists the 61 design sources and the depth of each read. These records let readers inspect the synthesis; their counts do not measure evidence quality.

The synthesis distinguishes sourced findings, design choices and hypotheses (Supplement S2.1). One author, assisted by AI models, made the mappings and coverage judgments; agreement between independent human coders has not been measured. Section 9.2 explains the limits this focused selection places on coverage and transfer.

The author used Anthropic Claude and OpenAI ChatGPT/Codex in the research. The models searched for and read the sources, proposed the mappings and comparison codes, ran the prior-work search, wrote the scripts, synthetic data and figures, and reviewed drafts. Every cited work was read in full, or, for a book or standard, the cited chapters; Supplement S1 records the version

and passages for each and names the few parts not read, such as reference pages. A check script recomputes the counts, lineage and arithmetic on every proposed change. The author decided what entered the paper. Model review is not independent human peer review.

4. From product activity to protective decisions

This section identifies the parties whose activity the records describe, the path from records to protective decisions along which meaning must be preserved, and the responsibilities that path requires. It begins the answer to RQ1, which Section 5 continues and Table 4 in Section 6.3 completes.

4.1 The platform and its participants

The proposed payment platform serves businesses, which serve end customers. A business holds a platform account. It may instead hold a connected account under another business that has integrated the platform into its product. A merchant is a business accepting payments through either account type. People act for businesses through operator accounts. Software agents may use delegated credentials for a business or end customer; these are distinct from the investigating agent.

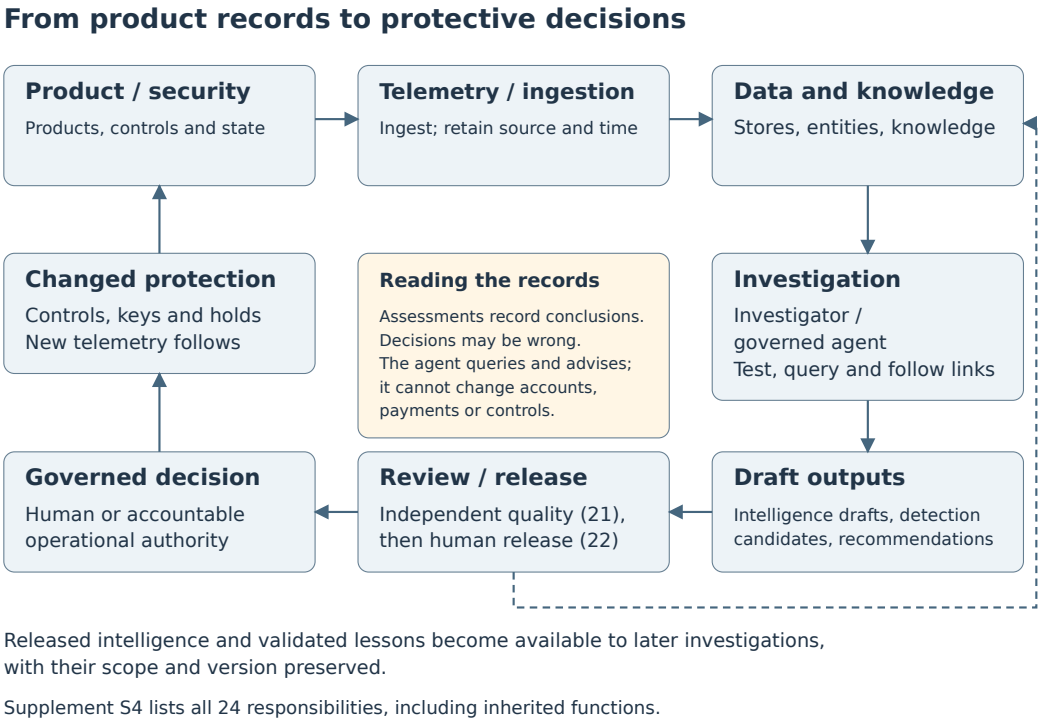
These roles determine whose activity a record describes. An end customer's action is not automatically an action by the business or its operator. The platform's products may span onboarding, identity verification, payments, subscriptions, payouts, treasury and risk controls. Businesses may also report their customers' registrations, logins and payments. The specification identifies what investigators need from these records without assuming a provider already collects or links them.

4.2 The path from records to action

Data pipelines collect telemetry and state records, map them to shared entity and field definitions, and preserve their origins. Storage and correlation services make them searchable by time and entity. Investigators use them to prepare intelligence, meaning assessments intended to inform decisions, together with proposed detections and recommendations. Authorized people or accountable operational systems decide whether to change controls, and those changes affect what later records can show (Figure 1).

Three feedback loops connect this work. **Investigative feedback** changes the next hypothesis or test within a case. **Organizational-learning feedback** makes released intelligence or tested procedures available to later cases as context. **Protective feedback** changes the platform through an authorized control or product decision. Their outputs require different authority and support different inferences.

Figure 1. From product records to protective decisions. Investigation produces drafts. Independent quality review precedes human release in the normal path. An authorized human may decide urgent containment before review under Section 6.4, without releasing the draft into organizational memory. Each resulting control decision records what the platform did, not whether the suspected abuse occurred.



The architecture separates three logical layers, called planes. They need not use separate databases:

- The **evidence plane** holds observations with their source, time and entities. Scores and control decisions belong here as observations of what a model concluded or a control did. Derived features belong here too, as values recomputable from recorded inputs; a feature computed over a cluster cites the cluster version it used, and the cluster itself remains a threat-knowledge item. A feature computed over a cluster that an agent proposed, or that depends on an agent output or released item, links back to that output or item and counts as dependent on it.
- The **structured threat-knowledge plane** holds revisable inferences, such as relationships between entities, groups of activity and conclusions with confidence and provenance.
- The **organizational-memory plane** holds released intelligence and tested lessons about procedure. These guide later work as hypotheses or context.

For example, two accounts using the same device is an observation. Common control is an inference. A released report retaining that qualified inference becomes organizational memory; retrieving it creates no new observation. An investigator may follow its lineage to cite the original observations at their original times and sources. A threat-intelligence reference model likewise traces the creator and modifiers of each intelligence product [51].

Dependence must also survive indirect reuse. A human label, assessment, control decision, detection or threat-knowledge item informed by an agent output links back to that output. Labeling, assessment, control, detection and threat-knowledge interfaces automatically attach every agent

output and every released organizational-memory item open in the writer's case or session. Only a person with release authority may remove a link, and removal is logged. Intelligence sent outside the platform carries an identifier. When content bearing that identifier returns through an external feed, the external-knowledge record links to the released item and counts as dependent on it.

These records remain dependent on the agent output that informed them. They cannot independently corroborate the agent's findings, a detection derived from its work, or the same hypothesis. The links capture only exposure observed by the interfaces: an unlinked record may still reflect something its writer read elsewhere. A missing link therefore does not establish independence. Detections that rely on agent-dependent labels are validated on external outcome channels or on cases worked by staff without access to agent outputs.

This separation cannot repair missing source data. An agent cannot retrieve telemetry never collected, reconstruct timing discarded by a pipeline, or establish an entity link without suitable identifiers.

4.3 Responsibilities and authority

Supplement S4 defines 24 responsibilities for the investigation system. They are functions, not proposed teams or a required number of agents.

Responsibilities 1 to 10 scope and conduct an investigation. They include human tasking, governed tools, the evidence store, hypothesis testing and entity relationships. Responsibilities 11 to 20 represent threat knowledge and turn findings into usable outputs: actor, technique and vulnerability knowledge, the structured threat-knowledge store, hunting for undetected abuse, detection development, threat-model integration, recommendations, finished-intelligence synthesis and evidence lineage. Responsibilities 21 to 24 govern use and reuse: independent quality review, human release, controlled learning and security of the investigation system.

Seventeen responsibilities were derived with payment-specific mechanisms or sources in view, six are inherited from threat-intelligence work, and controlled learning is adapted from agent-security research; Supplement S4 records each derivation. These origins describe how the responsibilities were derived, not uniqueness to payments. Payment authorization, payouts and account-review systems produce C1's decision records; their operational owners sit outside the investigation responsibilities.

Human tasking and release stay with people, and quality review remains independent of the work checked. Other functions may use one agent with tools, separate model sessions, several agents, fixed rules or a combination. Additional separation must justify its cost through narrower authority, access or failure scope.

The investigating agent may query permitted records and prepare intelligence, hunting and detection candidates, recommendations and proposed lessons. It cannot change accounts, payments or controls, certify its own output, or release conclusions into organizational memory.

5. Records an investigator can reason from

This section answers the records part of RQ1 in four steps: record classes that state what each record can establish (5.1), the abuse mechanisms that motivate particular records (5.2), the properties each store must preserve (5.3), and what published schemas already supply and privacy obligations permit (5.4 and 5.5). Table 4 in Section 6.3 adds the capabilities.

5.1 Separate observations from conclusions

A payment log, risk score and fraud report may concern the same transaction but establish different facts. The log records activity, the score a model's assessment, and the report what someone alleged. Copies of one report do not provide several independent confirmations. The record's class makes these limits explicit.

The same principle applies to relationships. A shared payout destination is observed; common control is inferred. Attribution, identifying a particular actor behind the activity, is a stronger claim requiring further support. An earlier attribution remains historical context when retrieved in a new case.

The specification uses seven record classes (Figure 2):

- **Source observations** record events or reports at their source.
- **Normalized observations** map source records to common entity and field definitions while retaining provenance.
- **Derived features** compute values from recorded inputs for specified entities and time windows.
- **Model or risk outputs** record assessments and the identity and version of the model.
- **Control outcomes** record decisions, independently of whether those decisions were correct.
- **Analytical inferences** state conclusions drawn from evidence.
- **Finished intelligence**, also called released intelligence, is analysis approved for reuse.

External knowledge is identified separately by source and type. It may be an observation, such as an exposed API key, an assessment, or a published catalog of abuse techniques. It supports what that source observed or described, not by itself a claim that abuse occurred inside the platform.

A class describes meaning, not reliability. A derived feature is only as reliable as its inputs and computation. An outcome label records who reported what and when; any separately established finding remains distinct.

Each normalized observation preserves its source and event identifiers, schema version, entities and lineage. It records event time and learned time, plus actor and request identifiers where available. Its environment distinguishes production from tests, synthetic data, replay or emulation. Its *tenant scope* identifies the business or other boundary within which the data may be accessed and used. Delivery identifiers prevent repeated delivery from counting or acting on one event twice.

Table 2 organizes the signals by family and class. Supplement S3 defines each signal and the inputs of every derived feature.

5.2 How abuse mechanisms motivate the records

Exposure and misuse require different evidence Measurements of exposed secrets establish exposure or validity at a particular time, not unauthorized use [58, 59] (Supplement S3). The specification therefore records exposure location and time as external knowledge, then compares credential use before and after exposure. Public visibility, validity, attacker possession and misuse remain separate propositions.

Timing can matter at minute scale [60, 61]. These observations motivate precise timestamps without establishing typical attack speed.

Figure 2. What each record class can establish. Counts refer to the 71 signals. Analytical inferences and finished intelligence are outputs, so neither appears among those input signals. Test, replay and emulation records are marked when created to distinguish them from production activity.

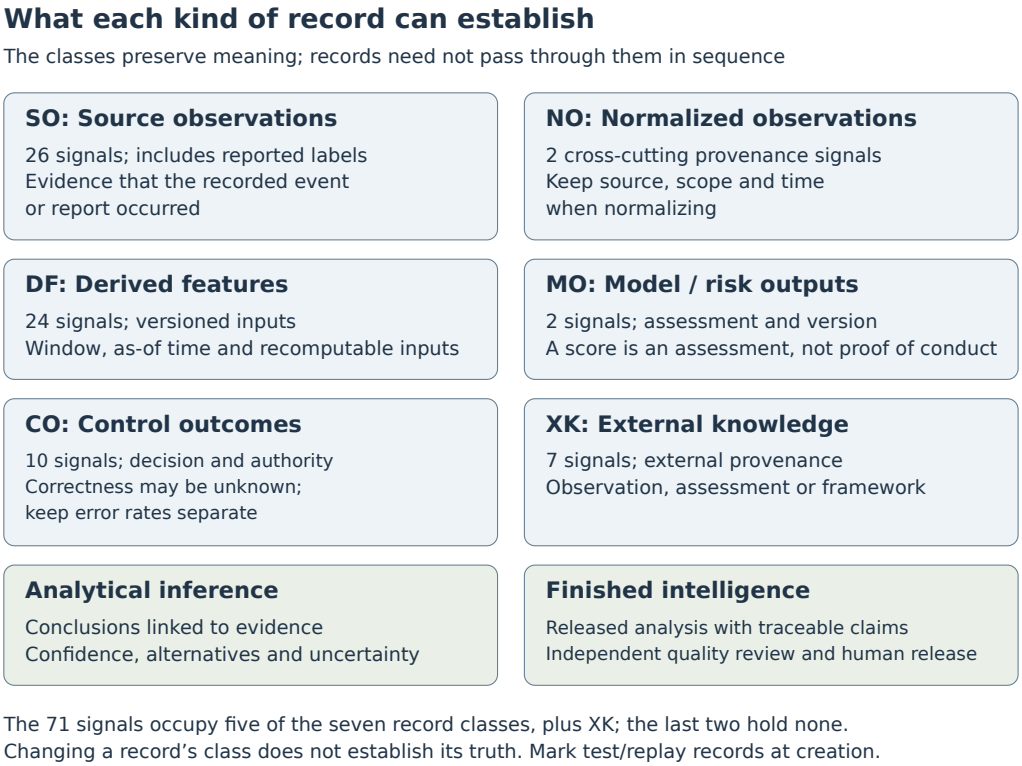


Table 2. The 71 signals by family. SO: source observation; NO: normalized observation; DF: derived feature; MO: model or risk output; CO: control outcome; XK: external knowledge. The final column counts partial public counterparts, cases where that comparison does not apply, and signals with no counterpart identified in the pinned schemas (Section 5.4). Token allowances permit another party to spend from a crypto wallet.

Family	Signals	Classes	Partial / n/a / none identified
Identity and onboarding	8	SO 3, DF 4, CO 1	3 / 4 / 1
Authentication, session and operator administration	10	SO 5, DF 4, CO 1	6 / 4 / 0
Credentials, API keys and request context	5	SO 3, DF 1, XK 1	2 / 2 / 1
Funds movement and payout destinations	11	SO 4, DF 5, CO 2	6 / 5 / 0
Payment attempts, authorization and payment controls	7	SO 2, DF 4, CO 1	3 / 4 / 0
End-customer lifecycle and entitlement	7	SO 2, DF 4, CO 1	3 / 4 / 0
Merchant storefront and business claims	2	DF 1, XK 1	0 / 2 / 0
Platform outputs and labels	4	SO 1, MO 2, CO 1	4 / 0 / 0
Agents, tools and approvals	8	SO 5, CO 3	4 / 0 / 4
Wallet signing and token allowances	2	SO 1, DF 1	0 / 1 / 1
External threat knowledge	5	XK 5	0 / 5 / 0
Provenance classes	2	NO 2	1 / 1 / 0

Access needs a history A login challenge is an extra verification step at sign-in. Research examines challenges triggered by unfamiliar devices and networks [62]. The design records authentication context, session issuance and later use. Applying the same lifecycle to onboarding links, which let businesses complete account setup, is a design analogy.

Changes after access can identify what an intruder attempted. In reported OAuth-application abuse, intruders used compromised accounts to create applications, grant them permissions, add credentials to new and existing applications and create inbox rules that hid messages from the user [63]. The design records these changes, with links to source observations. Recording changes to authentication factors and event destinations extends the same approach.

A device or browser fingerprint can help connect activity but cannot alone establish common control. One impersonation market sells stolen user profiles with browser fingerprint data and a browser extension meant to reproduce them. The study did not buy profiles to test that software [64]. The records retain that uncertainty.

Funds movements need destination histories The FBI's payroll-diversion advisory describes attackers changing where funds are sent [65]. Payout destinations therefore have their own identities and histories: creation, changes, first observation, validation and reuse across accounts. Trace A applies this analogy to a hypothetical key permitted to change a payout destination.

Regulatory guidance motivates measures of time between receipt and onward transfer, and concentration at receiving entities [66]. It does not validate a per-account detector.

Business descriptions need corroboration Storefronts, merchant accounts and statement descriptors, the merchant names cardholders see, need not map one-to-one. ScamMagnifier linked fraudulent storefronts through merchant identifiers, with a provider corroborating some links. Its checkouts stopped before purchase and did not observe settlement [67].

Studies of abusive merchants describe misdeclared business categories and movement between banks [68, 69]. FTC complaints allege pooling or spreading transactions across accounts to evade monitoring, and misleading representations [70–72]. The design consequently links storefronts, descriptors, account histories and verified ownership records. It retains declared business details with their source and time for comparison with activity. These allegations and links motivate investigation; they do not establish intent or ownership in a new case.

Labels need their reporting path An outcome label can be missing because its report is late or because a control prevented the outcome. It can also be present and false. Payment-specific theoretical work distinguishes these problems and proposes estimation methods under assumptions [9]. The specification uses those distinctions to define records, without assuming that an estimator recovers hidden platform outcomes.

Each report retains its type, source, lag and attributed transaction or entity. It keeps three times: when the reported outcome occurred, where known; when the report was made at its source; and when it arrived at the platform. Its channel supplies a reporting window and clock-start event. The evaluation separately records its chosen maturity period, how long it waits before reading a missing report as a negative. A period shorter than the channel's window, or any period for a channel without one, rests on a delay model (Section 7.1). Every decision that may have shaped the report remains linked, and there can be several. A dispute can follow a blocked attempt, a retry and an approval, and one account restriction can affect many payments. Each link therefore states the decision's role. The report's later status, such as withdrawn, reversed, won or lost, has

its own history, which records whether each won or lost status followed adjudication. A status change or correction is a new record that references the earlier one, so what the platform believed at each time can be reconstructed. Some channels never adjudicate a report, yet a provider may still record it as lost: Stripe's dispute guide says disputes that network rules or local regulations make unchallengeable are closed as lost when they open [73] (Section 7.1).

An estimate that a report is false belongs to the assessment store, linked to the report with the responsible model or reviewer and version. This matters when supporting material itself may be fabricated, as participants describe for refund fraud involving generative AI [74]. A report may be both late and false, so one status cannot replace the other.

End customers are not platform accounts A legitimate business may have abusive customers. One provider's public documentation describes multi-accounting, account sharing, free trial abuse, account farming and deliberate non-payment for accrued usage [75–78]. The customer-evaluations page describes its reuse reasons as reuse within one business over the past seven days; the account-farming page does not specify whether its links span businesses.

The specification records end-customer registration and billing histories, reused identifiers, simultaneous use and unpaid usage within a business. Linking an end customer's identity across businesses requires separate authorization, a permitted purpose, privacy oversight and supporting evidence. This restriction does not prohibit authorized fraud-prevention aggregation on instruments, devices or credentials under a stated and logged scope.

5.3 Preserve meaning across stores, time and relationships

The investigation must be able to retrieve an observation, its interpretation and the decision it informed separately. These distinctions concern logical records; they do not require separate database products. Table 3 lists the properties each store must preserve.

Write authority follows the record's meaning. Ingestion appends to the evidence plane. Correlation software and the agent may propose threat-knowledge items; a person or accountable process accepts them. Agent proposals remain marked context and cannot support current-conduct claims without their underlying observations. Only a person releasing a reviewed draft or a tested, reviewed lesson adds or changes a claim in organizational memory. Later status records for a released item, such as monitoring findings, expiry and withdrawal, add no claim and record who or what wrote them. Rule checkers and quality reviewers write their findings on each draft version to the agent-and-approval ledger.

Event time and learned time answer different questions. A relationship added today with an earlier effective date was unavailable to yesterday's decision. Revisions, external facts, graph membership and feature definitions therefore retain the versions used. A derived fact's learned time is when that result became available to queries, not merely the latest availability of its inputs. It cannot precede any input's learned time.

A decision's reproducible snapshot need not copy the entire store. The system logs the inputs used and retains versions from which other then-available inputs can be reconstructed. This supports checking both what the decider used and what could have been known.

Evaluation time differs from decision time. Later outcomes may evaluate an earlier prediction whose features remain fixed. The evaluation declares its outcome window and maturity period. Training and model selection use a separate cutoff: labels unavailable then cannot enter them.

Table 3. What each analytical store preserves. The full contract is in Supplement S4.1. These are requirements for the proposed platform, not claims about an existing provider. The structured threat-knowledge and organizational-memory stores form their own planes, and the other stores hold evidence-plane records. Clusters in the entity-relation graph remain threat-knowledge items. External-knowledge records support only what their source observed or described, and ledger entries only what happened during the investigation itself.

Store or shared requirement	Required properties
Observation log	Source records and revisions, identifiers, environment, tenant scope and lineage; separate event, source-recording, ingestion and query-availability times; repeated delivery does not duplicate the event
Entity-relation graph	Typed identifier links with source, namespace, effective interval, learned time, version, confidence, benign explanations, and permitted purpose and scope; versioned cluster membership; end customers scoped to a business
Feature store	Recomputable inputs and versions; decision-time features exclude later information; later labels use the evaluation's outcome window and maturity; features inherit their inputs' agent-output and released-item links
Label store	Report source, type and attributed entity; occurrence, report and arrival times; appended value history; typed links to every decision that may have shaped the report; links to separate truth assessments and any informing agent output or released item; report presence, channel state and query access kept apart
Assessment store	Model or reviewer identity and version; review selection route and probability; links to any informing agent output or released item; unknown, not-assessed and failed-open states kept distinct from low risk; failed-open means an assessment failed and activity continued
Control-outcome store	Decision, authority, reason and version; reproducible available inputs; alternative-decision probabilities where applicable; human assignment and what the decider was shown; links to any informing agent output or released item; effects on each outcome channel, fixed with the decision; for action before review, the urgent route, impact class and later review outcome; correctness reviews with their selection route, separate from population error estimates
Structured threat-knowledge store	Revisable relationships and conclusions with confidence, input and rule versions, writer and status; agent proposals marked as unreleased context; links to informing agent outputs and released items
Organizational-memory store	Human-released items with approved claims and their stated confidence, version, scope, expiry, reviewer findings, lineage and correction links; exonerating items have independent counterexamples, a re-test date and the evidence and cluster versions whose change expires them early; proposed lessons wait outside it with their counterexample tests, and released lessons keep monitoring and withdrawal records
External-knowledge store	Source, type, times observed and published, validity period, retrieval time and license; a link to the released item whose identifier the returning content bears
Agent-and-approval ledger	Model, instructions, inputs, tool calls; approver, binding, expiry and reuse rule of each approval; approved and executed payloads, tool-manifest version and hash at approval and at execution, memory writes, outbound-traffic decisions; rule-checker and reviewer findings per draft version; marked untrusted content and exposed agent output; logged agent-output and released-item link removals
Query coverage record	Gateway-written entities and intervals, requested and evaluated; as-of cutoff, schema versions, sampling, per-source watermarks, known gaps and retention horizon, access filtering and limits reached; derived result status; not writable by the agent
Shared time and graph requirements	Reconstruct what was available at each decision; preserve reporting clocks and delays, retention limits, account histories and report-to-transaction links

Applying the decision-time cutoff to evaluation labels would wrongly exclude the late outcomes being measured.

Channel state differs from report presence and access. For each affected item, channel and read time, the records distinguish an open channel, a temporary delay, a permanent closure and unknown state. A missing report matures only after a defined window that remained open and was fully covered. If a decision changed the channel's meaning, as a liability shift after successful authentication does (Section 7.1), a matured absence is read under that changed meaning and does not by itself support a negative for the original outcome. A channel without a defined window never supports a negative by maturity alone. A matured absence is provisional, not final. Network and bank rules can reopen or extend a window after it has elapsed, as when a monitoring program lets issuers submit fraud disputes past the standard deadline [79] or banks accept a late return [80]. Each such rule event is recorded as a channel-state transition with its source, rule version and learned time, although no platform decision caused it. A decision's effect on each channel is fixed when the decision is recorded; later transitions are separate records that never edit it. Inaccessible records remain unavailable without erasing a known channel state or an earlier report. Supplement S4.1 specifies the clock-start and transition fields, with further rules on program status, revised negatives and reopened reports.

An entity link is not a claim of common control. A verified identifier from the same namespace and relevant interval supports a stronger, or hard, link. A weaker, or soft, link may have several explanations: benign accounts can share a proxy IP address [45]. Both retain source and confounders, meaning plausible alternative explanations. Account-age records address aged accounts sold for abuse [81]. Where crypto products are in scope, wallet-to-spender links record permissions motivated by phishing that induces harmful transactions [82]. Supplement S4 defines each relationship's fields.

5.4 What public schemas reveal

This comparison shows how much of a required record a published interface already supplies and where the specification goes beyond it. It asks whether a proposed observation has at least a partial counterpart in a published object, field or event. Of the 71 signals, 32 have partial counterparts. The question does not apply to 32: 24 derived features, seven external-knowledge signals and one provenance signal. A keyword search of the three files identified no counterpart for seven, including request context, identity-capture session context and several agent observations. That absence is bounded by the terms searched.

A partial match can leave much of the required record unspecified. An onramp-session object, for buying crypto, and an event-destination configuration both record current state without a complete matching lifecycle event stream. The crypto documentation describes an onramp webhook, a notification sent to another system, for status changes after creation and none at creation [83]. Neither the event types that the pinned schemas declare nor their lists of webhook-endpoint events name a crypto or onramp event [57]. A request ID likewise supplies neither full request context nor the history of an onboarding link's use.

Supplement S3 records the classification; S5 provides schema pins, hashes, event extracts and exact field locations. These findings describe the inspected schemas, not the provider's private telemetry or every possible public interface.

5.5 Privacy and operating constraints

A deployment must specify retention and deletion for each record class before collecting it. Append-only logs preserve history but must still permit required deletion of personal payloads, derived data, backups and identifying lineage. Encryption-key destruction is sufficient only if recoverable keys and plaintext copies are also removed. Retained audit structure must not preserve information required to be deleted. Authorized holds record their scope and retention rule.

The agent-and-approval ledger uses references to card data, identity documents and other regulated data rather than copies. Tokens or keyed hashes may be used only where the applicable standard permits. The ledger should not become a new sensitive-data store or widen payment-card security scope.

Use of external model providers depends on their processing terms and permitted processing locations. Drafts concerning suspicious-activity reporting remain subject to the platform's confidentiality and subject-notification rules.

Cross-business linking requires a stated purpose and authorization for each identifier type. Permission to use a payment-instrument link does not automatically authorize linking devices or verified people. These deployment obligations constrain the specification; this paper does not determine which legal basis applies in a particular deployment.

6. How the agent investigates

This section answers RQ2. It follows one investigation from its assignment to the release of its outputs, then addresses the inputs that could subvert it. Table 4 also completes the answer to RQ1 by listing the agent's analytical capabilities with the limit on what each can establish.

6.1 Set the question and scope

A human assigns an investigative question or approves a standing task template. The assignment defines permitted data, entities, period, current period and outputs. Each run records the assignment, or the template and its version, that authorized it, and duplicate alerts are merged into cases before tasking.

The agent applies the record-class limits while reasoning. A score can direct attention without establishing abuse. A control outcome records a decision of potentially unknown correctness. A feature retains its inputs, computation period and data freshness. A label must be read with its reporting path and the decisions that shaped it.

Earlier intelligence may suggest a hypothesis or a useful first query. Retrieval returns its publication date, relevance period, underlying sources, confidence, restrictions and corrections where recorded. Several reports repeating one observation remain one source of evidence. Claims about current conduct require observations from the current period.

6.2 Test explanations and interpret the query that ran

An investigation asks which observation would distinguish plausible explanations. The agent converts that question into a permitted query, such as reconstructing a timeline or following an entity link. It first establishes what the available schemas and services mean.

The loop is **context → hypothesis → permitted test → observation → interpretation → revised hypothesis or conclusion**. Its stopping point may be an inconclusive assessment when the observations cannot separate the remaining explanations.

A complete result with no rows establishes only that no accessible rows matched. It counts against a hypothesis when the suspected activity should have produced that observation and the records cover where it would appear. Other results require different interpretations:

- A partial result supports no conclusion about the uncovered data or period.
- A truncated result may contain useful matches but cannot establish a population total.
- A denied query records an authorization decision.
- An unavailable dataset means the intended test did not occur.

The gateway that enforces the agent's scope (Section 6.5) also attaches the coverage record, independently of the agent. It records the requested and evaluated entities, with their tenant and identifier namespaces, the requested and evaluated event intervals, source and schema versions and any sampling. It also records the as-of cutoff, which limits the query to facts the system had learned by that time. For each source it records the event and ingestion watermarks, meaning how far event time and loading have progressed, known gaps and the retention horizon. A requested interval that extends past a source's event watermark, begins before its retention horizon or overlaps a known gap leaves that source's result partial, as does sampling of that source. It also records whether access filtering removed rows or row, time or cost limits were reached.

Fixed rules derive complete, partial, truncated, denied or unavailable status from that record; several statuses may apply. The agent can interpret results but cannot alter coverage fields. An unrecorded gap can still make a result appear complete, so recorded completeness is not knowledge of every possible loss.

The rule checker (Section 6.4) and the quality reviewer read the full coverage record. The agent sees only information allowed by its scope, including counts only when disclosure is authorized. A scope-limited flag identifies when authorized scope is narrower than the task's named entities, without disclosing outside records. The checker restricts absence claims accordingly. An unassessable hypothesis remains distinct from a query's status.

6.3 Capabilities and their boundaries

Table 4 lists the agent's permitted capabilities. Every use is scoped, limited and logged; none grants operational authority.

Entity links and labels require particular care. The agent retrieves each cluster's membership rule and version and tests benign explanations, including shared proxies [45].

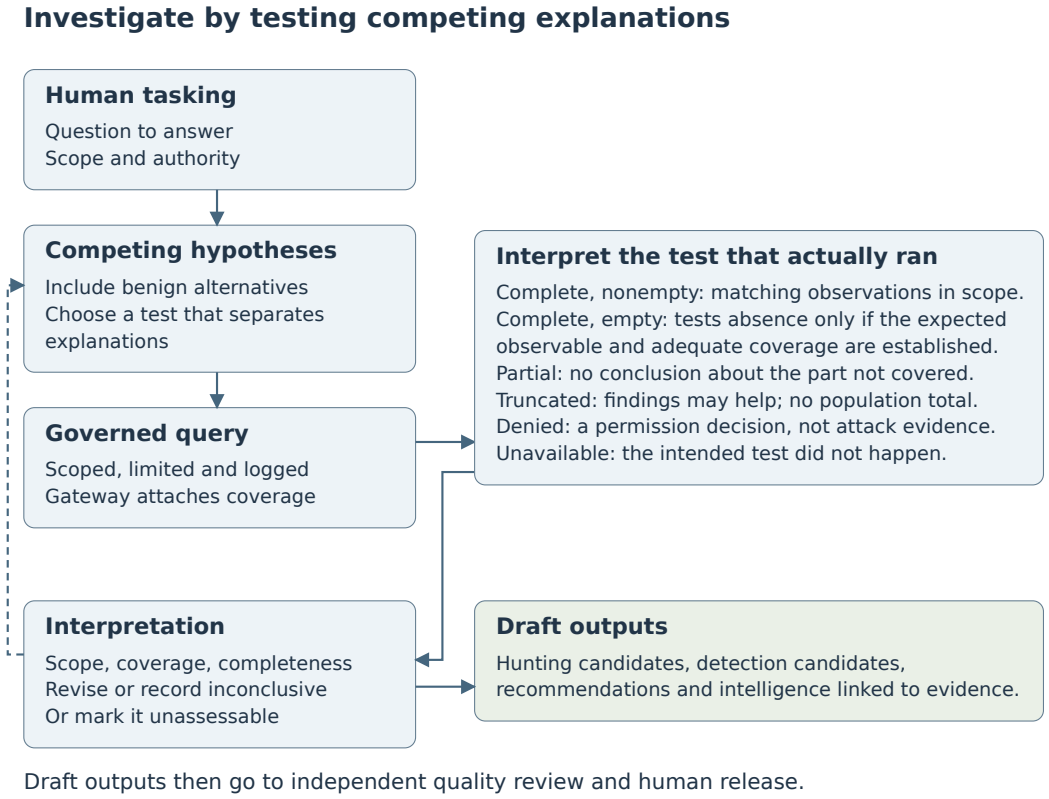
Likewise, no chargebacks may mean no abusive completed payments, blocked attempts, immature reporting or reports attributed elsewhere. Resolving a pre-dispute alert, an issuer notice that can be addressed before a chargeback, may also avert one. The agent examines decision gates, report times and transaction links before interpreting the absence.

A detection candidate states the behavior to detect, its inputs and an evaluation plan. That plan covers label provenance and how an adversary might adapt.

6.4 Check the claims, then authorize their use

A draft separates material claims from context. A claim is material when its falsity would change a recommendation, detection candidate or released statement about named entities. Each material

Figure 3. Investigation as a sequence of tests. Each query tests competing explanations within authorized scope. A gateway outside the agent attaches the query's coverage, which determines which interpretations remain supported. Draft outputs follow review and release under Section 6.4.



claim cites supporting records with their classes, event intervals and root observations, reached through lineage. It also lists the retrieved records that count against it, and quality review adds any it finds. The checker also identifies every record that the case's query runs returned within the claim's scope but that the draft neither cites nor lists against it; Supplement S4 gives how these records are shown and sampled. A claim that something is absent also cites the query runs whose coverage records support it. A claim that states or relies on an outcome that was not reported also cites, for each affected item, the state of each relevant outcome channel at the read time. The absence counts as a negative only where the channel matured with no report under its original meaning, and that negative remains provisional (Section 5.3). Claims about current conduct cite at least one source observation, normalized observation or derived feature whose event interval falls entirely within the task's current period. A score or control decision from that period does not meet this rule by itself, because it records an assessment or an action, not the conduct. A derived feature that carries an agent-output or released-item link does not meet the rule by itself either; the claim must also cite, for each named entity, a current-period observation among that feature's inputs.

A checker outside the agent enforces these rules before quality review. It rejects substitution of organizational memory for observations and deduplicates repeated records of one root observation. Distinct roots are not automatically independent: shared sources and common causes still need review. Supplement S4's draft schema gives the required fields.

Table 4. Permitted capabilities and their evidential limits. Task scope, permissions and resource limits apply throughout.

Capability	Information used	Boundary
Discover sources and schemas	Signal inventory, schemas of data stores, and records of counterparts in published API schemas	Finding a source does not grant access to it
Run governed queries	Permitted data stores, with scope limits, query limits and logging	Conclusions cover only the data and period the query examined. A denied query is not by itself evidence about the suspected abuse
Match records to entities and follow their links	Links labeled by type, known benign reasons for shared identifiers, and versioned clusters (computed groups of entities)	Propose relationships for assessment. A shared identifier does not by itself show common control. A cluster is the output of a computation, not an observed relationship
Reconstruct timelines	Credential exposure and later use, session issuance and later use, and account changes after access	Events close together in time do not prove that one caused the other
Interpret outcome labels	Report provenance, decision gates and links from reports to transactions	A missing report does not show that no abuse occurred
Generate hypotheses from published catalogs of abuse techniques	Mappings to a fixed version of each catalog	Naming a likely technique does not show that it was used
Draft hunting and detection candidates	Features, observations and outcome labels	A hunting candidate is a proposed search, not a finding. Detection candidates must still be validated under detection development and threat-model integration (responsibilities 16 and 17)
Recommend product or control changes	What each relevant control decision means, restriction options, what happens when a check fails, and protective actions	Authorized humans or accountable operational systems decide and act
Draft intelligence	Evidence with traceable lineage	Drafts require independent quality review and human release

Event timing also needs support. A time asserted by a business, customer or external source retains its source and uncertainty. Recent receipt does not establish recent conduct when the event interval is unresolved. Outcome windows instead use each channel's declared clock-start event.

Independent quality review examines the cited records directly, weighs contradicting evidence and alternative explanations, and checks attribution and fit to the intended reader. It verifies claims of detection validation against engineering evidence; responsibilities 16 and 17 still perform that validation. Reviewers and checkers are subject to the same untrusted-content safeguards as the agent.

When realistic volume forces reviewers to sample the cited records, sampling is tiered by materiality. Claims about named entities are checked in full, and every draft still requires human release. E8 tests this policy under the planned workload.

Independence is a hypothesis, not a property guaranteed by a model name. E6 and E8 measure whether review catches attacks and planted errors. E8 plants errors taken from the agent's own earlier runs alongside independently written errors and reports the catch rate for each. Separate sessions may share errors through common weights, and different models may share training data. A distinct method and organizational role can provide separation, but the reviewer must still examine the evidence.

Normal release follows quality review. The release decision records the person, exact version hash, claims with the confidence stated for each, scope, expiry and reviewer findings. The releaser sees supporting and contradicting records beside each material claim. Release preserves the assessment's uncertainty. Human release is itself exposed to automation bias, in which people follow an automated aid's incorrect advice or miss problems it fails to flag, and to the fall in detection when errors are rare [84]. In a laboratory search task, observers missed 7 percent of targets when targets appeared on half the trials and 30 percent when they appeared on 1 percent [85]. A test that plants errors often could therefore overstate how many the gate catches, so E8 plants errors at a realistic rate without telling reviewers or releasers which drafts are planted.

Urgent containment has a narrower route. A recommendation for a reversible action, such as a payout pause or API-key restriction, goes to an authorized on-call human with a decision target in minutes. Quality review follows within a set period, and an authorized human or accountable system decides whether to reverse the action. The draft does not enter organizational memory through this exception. Existing automatic controls remain in place.

Reversibility does not imply low impact. An API-key restriction may interrupt a merchant's integration. Actions are classed by impact as none, degraded service or outage; outage-class actions require notification and a restoration path.

Human review also creates a capacity limit. With assumed values of 20 drafts a day and 30 minutes per draft, review alone requires 10 hours daily. The system monitors release volume, returned drafts and the oldest queued draft. Urgent containment takes priority when the queue is saturated; expired drafts are discarded rather than released automatically. E5 measures whether the agent's contribution justifies this work, and E8 tests the gate under realistic volume and error prevalence.

6.5 Protect the investigation from its inputs

Businesses, customers and outside sources can supply content the agent reads: storefront pages, descriptions, refund evidence, retrieved documents and tool results. Indirect prompt injection uses such content to introduce instructions. Research demonstrates these attacks through external content and tool outputs [86, 87].

The architecture treats this material as data with no authority over instructions or tools. The gateway enforces task and tenant scope, least privilege, the allowed tools, query limits and outbound-traffic restrictions. These boundaries constrain access and action even if hostile input influences reasoning.

Approval binds to the operation actually reviewed. It identifies the exact payload, target, task, tenant, tool and tool-manifest version, plus expiry and permitted reuse. At execution, the gateway checks both the match and current permission. This addresses altered operations and changed tool descriptions [88, 89]. Approval of an investigative operation cannot grant the agent authority over accounts, payments or controls.

Memory and outbound traffic have separate controls. Proposed lessons remain outside organizational memory until testing, quality review and human release. This responds to documented

memory-poisoning routes [90]. Traffic restrictions respond to demonstrated exfiltration, in which injected instructions passed user data to an attacker through search queries or links [86]. Testing includes adaptive attackers because fixed-test success can overstate resistance [91].

The threat model includes four attacker positions: an external abuser controlling retrieved content; a merchant also supplying decisions and refund evidence; a compromised operator; and an insider with approval or release authority. Each plane's write rules must be evaluated against all four. No control in this design stops the fourth position from releasing a false claim or removing an agent-output or released-item link, because its authority permits both. Against that position the safeguards are the release record, the logged link removal, expiry and re-testing of exonerating items, and correction (Section 7.3). They can detect and limit such actions, but they do not prevent them.

One consequential attack produces a plausible but false benign explanation. Poisoned evidence might suggest a legitimate merchant-of-record arrangement, where one business sells on behalf of others. An honest agent could then propose a lesson steering later investigations away from that infrastructure. Human release alone does not defeat this attack.

Released benign assessments and exonerating lessons therefore expire on a fixed date, or earlier when their evidence or cluster membership changes. They are re-tested against independent counterexamples from closed cases or reviewers who did not write the lesson. Labels, assessments and decisions informed by them retain links so they can be revisited after expiry or retraction.

A second attack reaches a person investigating the agent itself, through hostile content copied into the ledger. Untrusted status must survive storage and normalization. The ledger distinguishes copied untrusted content, agent output exposed to untrusted content, and agent output without that exposure. Exposure takes precedence when outputs also use other evidence. Ordinary source observations retain their own trust marks and are not assigned an agent-output category.

Ledger copies and agent outputs may suggest hypotheses, but claims cite the underlying evidence-plane records. Their promotion into organizational memory still requires independent testing and human release.

The ledger makes the agent itself investigable. For each tool call it retains the provenance of inputs reaching the model, approved and executed payload hashes, memory writes and outbound-traffic decisions. It also records model identity, version, instructions and decoding settings, with record classes intact.

The architecture does not include live adversary emulation. Any future use would require separate ethical, human and operational authorization, enforced scope, tamper-evident logs and test-data provenance.

7. Interpreting feedback after protective decisions

This section answers RQ3. Section 4.2 distinguished three feedback loops. This section concerns the two that leave the case: protective feedback, which changes the platform and therefore the evidence available afterwards (Sections 7.1 and 7.2), and organizational-learning feedback, which carries conclusions and procedures into later cases (Section 7.3).

7.1 Separate delay, observability and report truth

A control changes both activity and the evidence later available about it. Blocking a payment prevents that attempt from completing; holding a payout may delay it. The absence of a later outcome therefore cannot be interpreted without the decision that shaped its channel.

C1 keeps three questions separate (Figure 4): **Could a report still arrive? Could this outcome occur under the decision taken? Is the report true?** Their answers can depend on one another. A report may be late and false, while a control may close one channel and leave another open.

Delay: a report has not arrived yet A deployed payment-fraud system trains on user fraud reports, which arrive after the transaction [6]. The relevant reporting window depends on the channel. A delay model can distinguish a pending outcome from a negative after sufficient time, as in Chapelle's conversion model [12]. Reading immature negatives as final can reverse a comparison: the illustrative provider's engineers report a fraud model trained on far more payments that looked better at first, until, as more fraud reports arrived, its measured performance fell below the earlier model's and it was rolled back [39]. A channel without a defined reporting window, such as a victim report, supplies no such maturity bound.

The records preserve arrival time, channel clocks and the evaluation's maturity choice. A late report may evaluate an earlier fixed prediction. It must not enter that prediction's features or a training run whose cutoff preceded its availability.

Observability: the decision changed the possible outcome A payment permanently declined before completion cannot generate a chargeback on that attempt. It may still produce a complaint or incident finding. A payout hold changes when funds move, not when a dispute can arrive.

Figure 4. How decisions shape later evidence. Delay concerns report arrival; unobservability concerns outcomes a decision prevents; false reports concern truth. Separate fields preserve these distinctions, while evaluation must justify their joint assumptions. Corrected intelligence and tested lessons return through review and release.



Stripe's dispute guide says card networks typically allow cardholders to dispute within 120 days of the original payment, sometimes longer, and that for a future event or service the window generally starts on the event date [73]. Rules vary by network and version. The records therefore distinguish authorization, capture and processing times and use the clock-start event specified by the applicable channel rule. Missing rules or timestamps leave maturity unknown.

Capture may also gate possible-fraud reporting. Stripe's monitoring guide says issuers must report possible fraud on captured payments, even after refund, but are not required to report it on authorizations. It adds that an authorization reversed before capture is not reported [79]. Stripe's dispute guide adds that a refund processed as a reversal, which usually happens within about 2 hours of capture, can also prevent a fraud report [73]. Capture alone therefore does not fix that channel's state. Thus a review ending in reversal can close that channel while other incident reports remain possible.

A control can change a channel's meaning without closing it. Stripe's 3-D Secure guide says liability for eligible fraud disputes can be expected to shift to the issuer after successful authentication, but is not guaranteed. It adds that an early fraud warning, a network report of suspected fraud, may still arrive [92]. Authentication result, reported liability status and each channel's state must therefore remain separate.

Payout timing is recorded against the relevant reporting windows because funds may leave before reports arrive. The effect is recorded per channel: closed, delayed, changed in meaning, none or unknown.

These channel properties come from card-network rules, and other rails differ. On the illustrative provider's bank-debit rails, a payer's dispute or return is honored without network adjudication or appeal. A SEPA Direct Debit dispute within eight weeks of the debit is honored with no questions asked, and such disputes are final, with no process for appeal [93]. After eight weeks and up to 13 months, a customer can dispute only a debit considered unauthorized, and the customer's bank decides; the window that applies therefore depends on the claim, and an absence at eight weeks is not a negative for unauthorized-debit fraud [93]. Inside its window, an ACH return is refunded without question, and the ACH Network has no appeals process of its own [80]. A report honored without question is never adjudicated, so any won or lost status recorded for it says nothing about its truth, which rests entirely on a separate assessment. A SEPA dispute honored with no questions asked is not by itself a fraud allegation. For ACH returns, the provider's terms give 60 days for consumer accounts and 2 days for business accounts, counted from the customer's bank statement [80], while the Nacha return deadlines that banks apply run from the settlement date of the original debit [94]; a channel record therefore names the rule from which it takes its clock start. Banks may also accept late returns outside the window [80]. The product scope also lists instant push payments and stablecoin payments (Supplement S2); their reporting channels, if any, follow each rail's own rules and windows, which this paper did not review. Each channel record therefore names its rail, the attributes its window depends on and whether the channel adjudicates report truth.

Truth: the report may be wrong Bad-faith disputes and fabricated refund evidence can make the adversary a source of outcome labels [9, 74]. One evaluation interface accepts the business's decision on each evaluation (allowed, restricted or blocked) and says these reports train its models [75]. Such reports may also carry that business's errors. Models trained on these labels predict reported outcomes unless the reports' relationship to truth is separately established.

Report type and source therefore remain attached to the label. Any judgment that it is false is a separate assessment with its own provenance. This preserves both the report and the uncertainty about its truth.

What the records permit an evaluation to estimate Selective-label methods and causal analysis require more than the outcome field [1, 2, 4]. For each decision, the specification records available inputs, the score, threshold or rule, policy version and alternative-decision probabilities where the policy is not deterministic. Human decisions also record the decider, case assignment and information shown.

If all decision inputs are logged and remaining decision variation is independent randomization, conditional exchangeability holds given those inputs. In practical terms, the logged history must account for variables influencing both decision and outcome [4]. Unlogged inputs used by a decider break that guarantee and must be identified. The records support examining assumptions; they do not validate an estimator or the bounds in theoretical payment models [9, 10].

Three target quantities illustrate the distinction:

- **Candidate-policy value:** the average outcome a proposed decline policy would produce on a stated measure. Estimation from another policy's logs needs decision inputs, probabilities and overlap for the decisions the candidate would take.
- **The chargeback rate if all cases were allowed:** this needs comparable cases with some chance of being allowed. A class always blocked supplies no allowed comparison.
- **A control's error rate:** this needs a defined cohort, outcome window and denominator, with correctness reviewed for every case or a random sample with recorded selection probabilities. Appeals alone do not establish a cohort rate.

Comparing human deciders of different leniency additionally requires random case assignment for the cited evaluation approach [1]. Without further assumptions, outcomes hidden by deterministic blocks support ranges rather than a recovered comparison.

An observational evaluation must define its target population, the decisions it compares and the outcome, then justify the assumptions linking observed decisions to alternatives [4]:

- **Consistency:** each compared decision is precisely defined, and a case's observed outcome equals its outcome under the decision actually taken.
- **Conditional exchangeability:** after conditioning on logged history, cases receiving different decisions have no remaining differences that bias the comparison.
- **Overlap:** comparable cases had a nonzero chance of each relevant alternative decision.
- **No interference:** changing one attempt's decision does not change other attempts' outcomes, including retries.

Repeated decisions require the relevant conditions at each step because earlier decisions can change later inputs [4]. Interference is especially consequential when blocked actors move to sibling accounts or other merchants. Links from each report to every decision that may have shaped it make these dependencies visible; they do not remove them.

Delay and false reports add assumptions of their own. For delay, an evaluation specifies a distribution and period of stability. It assumes that, given recorded case attributes and channel, reading time is unrelated to when the report would arrive. For false reports, a noise model specifies error rates by decision, channel and reporter where relevant. Rates come from a random sample reviewed for truth or are stated as bounds.

These assumptions are joint. Allowed payments may receive chargebacks while blocked items receive investigator labels or complaints, with different delays and clocks. Separate fields make these differences visible; they do not make the observation processes independent.

Obtaining missing comparisons through prospective exploration would mean allowing some attempts the existing policy would block. That requires separate ethical, human and operational authorization. It is neither performed nor authorized here.

Attribution and resolution change measured ratios Account structure affects where a report is recorded and which transactions a ratio describes. FTC complaints allege pooling or spreading activity to evade monitoring [70, 71]. These allegations motivate transaction links and cluster-level measures, without establishing erroneous attribution in a new case.

The same outcome definition, cohort and window must be retained when comparing account and cluster ratios. A pooled cohort ratio is a weighted average and cannot exceed the highest account ratio merely through pooling (Section 8.2). It can conceal a high account ratio among lower ones.

Resolution also changes what becomes a chargeback. One FTC complaint alleges refunding on pre-dispute alerts to avoid chargeback recording; the resulting order was stipulated [71]. Stripe's dispute guide says a business can proactively refund a payment flagged by an early fraud warning to prevent a dispute, while advising against refunding every flagged payment. It adds that such a refund does not affect the fraud warning [73]. A lower chargeback ratio after alert resolution therefore does not establish less abuse.

The records retain each alert, its resolution and any later chargeback. Transaction links allow a defined measure to count each affected transaction once. Table 5 summarizes the resulting requirements.

7.2 Measure control errors and user costs separately

A known control decision can have unknown correctness. A study of five commercial identity-verification products reported false negative rates of 10.5 to 53.1 percent, the share of genuine participants' verification attempts that were rejected [95]. It did not report fraud-detection results. Those cohort results cannot become probabilities that an individual platform decision is wrong.

Denominators matter as much as percentages. A login-challenge study reports 47.8 percent same-session success for challenged sign-ins from unfamiliar devices and networks. It also reports 97.3 percent eventual access within a week. Its results count that share over users who failed a challenge, not over sign-ins, while its introduction states it over users who saw a challenge [62]. Its abstract treats the complement of the first rate as legitimate users failing to sign in, but its results note that hijacking activity depresses per-session success. Subtracting the first rate from 100 percent therefore does not give a clean legitimate-user failure rate.

The specification links a control decision to a correctness review where available and separately to applicable population error estimates. Each estimate retains its version, cohort, outcome window, denominator, uncertainty and scope. Evaluation weighs detection gain against legitimate-user cost. Unknown, not-assessed and failed-open states remain distinct from low risk [77].

Controls also need retesting as adversaries adapt. Published examples include changed card-testing methods, aged accounts, banks changed after payment disruption, and filtering of recognizable test transactions [69, 81, 96]. Adaptive prompt-injection attacks likewise expose limits of fixed tests [91]. Responsibilities 16 and 17 own this evaluation.

Table 5. Requirements for interpreting feedback. Each response follows from the stated problem and evidence basis.

#	Problem	Evidence basis	Required response
1	A decision can close or delay an outcome channel, or change its meaning. Effects differ by channel; payout holds do not delay disputes.	Platform documentation that capture gates fraud reporting and that authentication can shift dispute liability; a theoretical payments preprint; established principles for inferring outcomes under a different decision	Record the decision and its effect on named outcome channels; state the assumptions needed to estimate unobserved outcomes
2	Reports can arrive late; this paper does not establish maturity periods	Theory and deployed-system evidence of delay	Preserve reporting clocks and arrival times. Reconstruct decision inputs as of the decision. Apply the evaluation's separate outcome window and maturity period to later labels.
3	Some reports or their supporting evidence may be fabricated	Theory and interview research	Keep report type and source with the label; keep any estimate that the report is mistaken or deliberately false as a linked assessment, not a field of the report
4	Controls can wrongly restrict or block legitimate users	Peer-reviewed evaluations	Keep decisions, reviews of their correctness and population-level error estimates separate; weigh detection gain against the cost to legitimate users
5	Adversaries can adapt to controls and testing	Platform publication, peer-reviewed measurement and preprint	Test adaptively within responsibilities 16 and 17; keep test traffic hard for adversaries to recognize but identifiable within the system
6	Decisions a business reports to the platform may carry the business's errors into platform labels	Platform documentation of business-reported decisions (Section 7.1); inference that they may carry errors	Keep decisions and outcomes reported by businesses distinct from platform labels
7	Account structures can hide which transactions and entities a report concerns	Enforcement allegations	Keep the links from each report to its transaction and entities; compute defined metrics for each cluster of related accounts
8	Resolving a pre-dispute alert may avert a later chargeback	Platform documentation that a refund can prevent a dispute; enforcement allegations for the misuse	Record alert resolutions separately; do not read a lower chargeback ratio, by itself, as less abuse
9	A risk assessment may return a not-assessed result or fail open (let the activity proceed)	Platform documentation	Keep unknown and failure states distinct; do not convert either into low risk

Test, replay and emulation records are marked when created and excluded from production labels, intelligence and lessons unless their evaluation use is explicitly authorized. The mark remains internal so it does not help adversaries identify test traffic.

7.3 Correct conclusions and test lessons before reuse

New evidence may require correcting or retracting released intelligence. The correction links to the original, retains the new evidence's provenance, repeats applicable review and release, and notifies downstream users where warranted. Threat-intelligence lifecycle research likewise lets an entity that receives intelligence enrich it as a new version or retract it [97].

A reusable procedure follows a different path. A productive query, a useful move between related entities, or a recurring benign explanation becomes a proposed lesson. It enters a queue for prioritization and independent tests against counterexamples. Lessons that exonerate expire and are re-tested.

After testing, quality review and human release, a lesson may be reused within a defined scope and version. It remains monitored and can be withdrawn. This is controlled learning, responsibility 23.

Memory-poisoning and experience-reuse studies show how planted or mistaken entries can steer later work [90, 98]. A validated lesson therefore guides procedure; it is never evidence that the suspected activity occurred in the current case.

8. Two hypothetical investigations

The following traces apply the specification to fixed hypothetical observations. Each asks what happened, which explanations remain plausible, and what further test would distinguish them. Their purpose is to expose the consequences of missing context. Supplement S6 supplies the synthetic transaction records and reproducible arithmetic.

8.1 Trace A: an exposed key and a changed payout destination

A repository scan finds a publicly visible API key. A human asks whether it enabled account compromise and payout redirection. The trace concerns a key not yet revoked, or activity before revocation; existing automatic revocation is outside this workflow.

Exposure alone does not establish that a secret is still usable [58]. It also does not show misuse. The agent therefore compares earlier requests with those after publication. A hypothetical repository record supplies publication time, separately from the later scan-detection time. Available earlier requests share a cloud-provider region and stable client fingerprint.

At nineteen minutes after publication, requests arrive through a residential proxy range, using a different fingerprint. They access account-profile and configuration endpoints. Minute-scale investigation is motivated by the deliberate exposure experiment [60], but developer testing or an internal scanner remains a possible explanation.

The same burst creates a webhook endpoint, changes the payout destination and accelerates the payout schedule. A webhook receives the platform's event notifications. This trace assumes the key permits these changes. If destination changes instead require a dashboard session and additional authentication, the investigation would need evidence of session takeover. The payroll-diversion advisory motivates the destination-change mechanism by analogy [65].

The next day, delayed records show that the same verified destination identifier was attached to two other accounts before exposure. The matching namespace and interval support destination reuse,

not common ownership or malicious intent. The agent checks account histories and ownership records for alternatives such as a legitimate business with several accounts or an intermediary receiving client funds.

Two limits change the conclusion. First, a requested seven-day baseline contains only two hours of earlier history. The change at nineteen minutes cannot be called the first misuse. Second, a detection already running on the platform froze its compromise assessment thirty minutes after publication, before the destination link was learned the next day. The link can inform the later investigation but cannot enter that earlier assessment.

The investigation prepares four outputs: a search for similar cases, a detection candidate linking exposure to use and destination changes, an intelligence draft, and recommendations for authorized action. A reversible payout pause or hold may follow the urgent path when the accelerated schedule appears. Credential rotation or revocation follows normal review.

Stripe's public risk guide describes pausing, delaying or holding connected-account payouts and extending payout schedules [99]. It does not define a pause-versus-hold distinction, so the specification treats both as reversible restrictions while recording which was applied. A deployment must confirm from its own permission model which credentials can change payout destinations and schedules, and what revocation would interrupt.

The detection candidate needs an appropriate outcome. A payout moves an account's balance to its payout destination, which here may be controlled by an attacker. It is not a card payment that can produce a chargeback. Possible outcomes include victim reports, returned or recalled transfers and confirmed incident findings. A permanent block prevents outcomes requiring completion; a hold may only delay them.

Victim reports and incident findings have no defined reporting window here. Those channels can supply confirmed positives but cannot reach "matured with no report." Evaluation must either justify a delay model, as conversion research does in its setting [12], or leave negatives unconfirmed. The recorded decision and channel states determine which interpretation is available.

8.2 Trace B: linked storefronts and dispute activity

A risk assessment and a rejected application identify similar storefronts. A human asks whether newly onboarded merchant accounts form an operation structured to evade dispute monitoring. The assessment selects a lead; the investigation must establish the underlying relationships.

Following the merchant-identifier approach used in storefront research [67], the hypothetical investigation links eleven storefronts sharing two templates and a descriptor pattern. They settle through four merchant accounts. A principal, a person who owns or controls a business, also appears on a closed account and the rejected application. Research and enforcement allegations motivate examining such returns after closure [68, 72]. Settlement relationships are stipulated here, not inferred from the cited study's uncompleted checkouts.

The next question is which transactions the outcome reports concern. Let T_i be the size of account i's fixed transaction cohort and D_i its distinct transactions with chargebacks. For disjoint cohorts with positive sizes and the same outcome definition and window:

$$r = \frac{\sum_i D_i}{\sum_i T_i} = \sum_i w_i r_i, \qquad r_i = \frac{D_i}{T_i}, \qquad w_i = \frac{T_i}{\sum_j T_j}.$$

The pooled ratio is a transaction-weighted average. Its nonnegative weights sum to one, so it lies between the lowest and highest account ratios.

The synthetic example gives accounts A to D 100 completed transactions each. After linking reports to transactions, every account has two chargebacked transactions: `2/100 = 2%`. The pooled chargeback ratio is `8/400 = 2%`.

Fourteen transactions also carry pre-dispute alert-resolution flags. Two overlap with the eight chargebacked transactions. A separate chargeback-or-alert-resolution measure therefore counts `8 + 14 - 2 = 20` transactions, or `20/400 = 5%`. Adding flags without deduplication gives the incorrect `22/400 = 5.5%`.

On that broader measure, A and B each have `8/100 = 8%`; C and D each have `2/100 = 2%`. The pooled 5 percent conceals the higher values at A and B. On the chargeback measure, all accounts remain at 2 percent, so pooling hides no higher rate. Neither measure is a fraud rate, and the example assumes no monitoring threshold.

Attribution changes which account appears affected. The chargebacks on D001 and D002 are recorded at account-X, outside the four cohorts. Counting where reports were recorded shows D at `0/100`. Verified transaction links restore D to `2/100` without changing the 400-transaction denominator or counting either report twice.

Such a mismatch can be legitimate. In Stripe's Connect documentation, a dispute on a direct charge debits the connected account's balance. A dispute on a destination charge or on separate charges and transfers debits the platform's balance, even when the connected account is the business of record, and the platform can try to recover funds by reversing the transfer [100]. Merchant-of-record and payment-facilitator arrangements can therefore explain a report appearing elsewhere. A missing transaction link leaves the affected measure incomplete, rather than justifying an assumed attribution. Reports concerning transactions outside the four cohorts are excluded.

A cohort ratio differs from a network-program ratio. Stripe's monitoring guide describes Mastercard's Excessive Chargeback Program as dividing one month's chargebacks by the previous month's captured payments and applying a minimum monthly chargeback count. It describes Visa's Acquirer Monitoring Program differently: each month, Visa divides the previous month's disputes plus early fraud warnings by captured payments in that same month, and it also sets thresholds on the count of those reports and, in one region, on their total value. The guide says both networks assign a report to the month in which they receive it, not the month of the original payment. For that Visa program, a transaction that appears in both the dispute report and the fraud report counts twice, and disputes resolved through pre-dispute products are excluded. Visa identifies a monitored account by the static part of its statement descriptor and its acquiring bank [79]. The guide describes itself as a general guide rather than a comprehensive reference, so E3 applies each program as the network's own documentation states it, such as Visa's program overview [101], and uses the guide only as a secondary summary. That overview defines the ratio over card-not-present VisaNet transactions, with settled transactions as the denominator, while the guide speaks of captured payments; E3 follows the network's definition [101].

Thus resolving alerts can lower the program ratio, and growing volume can lower a period ratio while reports are still arriving. Splitting activity can also keep accounts below a count threshold even when their ratios are identical. Detecting that pattern requires cluster-level counts, not merely a pooled cohort ratio.

Small cohorts leave substantial uncertainty. Under independent trials with a common rate, exact 95 percent intervals are 0.24 to 7.04 percent for 2/100, 0 to 3.62 percent for 0/100, and 0.87

to 3.90 percent for 8/400. For 8/100, the interval is 3.52 to 15.16 percent. Fisher's two-sided test comparing 8/100 with 2/100 gives p = 0.10. Cohorts of 100 transactions cannot reliably separate 8 percent from 2 percent. Repeated customers or a shared operator may correlate transactions, making those intervals and the test overstate precision.

The linked storefronts still require an explanation. Shared templates and descriptors may reflect a legitimate multi-brand business or aggregator. Verified identifiers support specific relationships without establishing intent or common control. The agent may produce a qualified cluster assessment, an inconclusive finding, a proposed account review or a detection candidate. It cannot reject or terminate accounts.

The enforcement sources remain allegations [70–72], and storefront content remains untrusted input. The synthetic rows contain no report-arrival dates. The ratios therefore assume that every report on the four cohorts has arrived, and the example cannot show reporting delay.

8.3 What retaining the context changes

Table 6. Five conclusions that depend on recorded context. Both columns use the stipulated observations. The comparison is logical, not a measured baseline.

Question and fixed observations	If context is omitted	If context is retained
Did misuse begin after exposure? A seven-day query returns two hours of earlier history; changed requests appear at +19 minutes.	The lack of an earlier returned match does not distinguish absent activity from missing history.	The baseline is partial. The change is observed, but the onset of misuse remains inconclusive.
Could the first assessment use destination reuse? The link was effective before exposure, learned at +1 day; the assessment was frozen at +30 minutes.	Effective-date ordering can introduce information unavailable at the decision.	Learned time excludes the link from that assessment. It may inform a later investigation; later labels may still evaluate the fixed assessment.
Can a blocked payout receive a negative outcome label? The transfer never completes.	The absence of a return or victim report can be mistaken for a benign outcome.	The return-or-recall channel is closed, not negative. Victim-report and incident-finding channels remain open. The blocked case remains in any target cohort covering all attempts.
Which transactions do reports concern? Four cohorts contain 400 transactions, eight with chargebacks; D001 and D002 are recorded at account-X. Two of fourteen alert resolutions overlap.	Recording-account counts show D at 0/100. Adding report flags gives 22/400 and changes the outcome definition.	Transaction links restore D to 2/100 and remove double counting. Chargebacks are 8/400 = 2%; the broader measure is 20/400 = 5%, with A and B at 8/100 and C and D at 2/100. Neither establishes fraud.
Does an earlier released assessment corroborate the cluster finding? It concerns the same descriptor pattern.	The conclusion can become apparent independent support for itself.	The assessment supplies a hypothesis or context. It adds no root observation and no independent corroboration.

Two rows of Table 6 add explicit hypothetical conditions: Trace A's payout is permanently blocked, and Trace B retrieves an earlier assessment of the same descriptor pattern. Omission does not force an error; it removes information needed to rule the error out.

For the blocked payout, excluding the case is valid only for a rate explicitly restricted to completed transfers. Estimating outcomes over all attempted payouts retains it in the target population and must address its unobserved outcome.

The examples identify a starting subset of records: the eleven signals behind Table 6, their lineage inputs and the storage requirements (Supplement S6).

The first two rows need request coverage, credential exposure, destination history and learned times. The second row also needs the frozen assessment and its freeze time, which the assessment store holds; none of the eleven signals records it. The third needs the control's effect on each channel. The fourth needs verified report-to-transaction links, deduplication and versioned cluster membership, which Trace B builds from storefront-to-merchant links. The fifth needs the separation between observations and organizational memory. These examples explain the required context without claiming that each named signal is uniquely necessary.

That context costs storage, query work, controlled cross-account access and investigator time. E1 tests the context in the first two rows of Table 6, E9 the channel states in the third, E3 the report linkage in the fourth, and E5 the human work the agent creates. The exploratory E2 and E4 bear on C1's decision links, and E2 also on late and false reports (Section 9.1). E8's planted drafts that rest a material claim only on a released item test the rule in the fifth row. The traces establish what the records mean. Those evaluations test whether some of the records improve investigation; none measures the storage, query work or controlled cross-account access that collecting them requires (Section 9.1).

9. Evaluating the specification

9.1 Questions, comparisons and failure criteria

The specification is worthwhile only if its added records and safeguards improve investigation enough to justify their cost. The evaluation plan separates the value of the records from the benefit of using an agent.

Supplement S7.1 gives each hypothesis, design, primary measure and proposed failure criteria. E1, E3, E8 and E9 are confirmatory; the others are exploratory. E3 tests the report-to-transaction links on which C1's outcome records depend, together with cluster-level monitoring. Its pass rule compares linked clusters with accounts as their reports are recorded, so a lead from clustering alone can pass it; only the third arm, which applies the rule to each account after linking, shows whether linkage contributes. E9 exercises C1's links from outcomes to control decisions, and its separation of late from unobservable outcomes, only jointly through the channel-state reading. No confirmatory evaluation tests the separate field for false reports or the decision links' other uses, such as estimating outcomes under other decisions. The exploratory E2 bears on the decision links and, through its sensitivity analysis, on late and false reports; E4 bears on the decision links. Passing E1, E3, E8 and E9 would therefore not establish the value of each of those links and fields on its own. E8 tests C3's release gate. The exploratory E6 checks that required links back to agent outputs exist. No evaluation tests the urgent-containment route in Section 6.4. The rate at which later review reverses its actions, and the service those actions interrupt, remain unmeasured. The questions below state what each evaluation is intended to resolve.

E1. Coverage and learned time: Do these records reduce unsupported conclusions in cases with incomplete history or links learned late, without reducing the correct, supported conclusions investigators reach in those cases or in cases with complete history?

E2. Unobserved outcomes: For a defined outcome and population, which identifying assumptions (the assumptions needed to estimate outcomes that could not be observed) are defensible? Where is overlap (Section 7.1) absent? How sensitive are estimates to late, mistaken or deliberately false reports?

E3. Report linkage and cluster monitoring: Does linking reports to transactions and applying the network's monthly rule, its ratio, its minimum count and any minimum value, to each cluster flag evasion earlier than account-level monitoring? A broader measure, such as Section 8.2's chargeback-or-alert-resolution measure, needs a separate evaluation.

E4. Measuring user harm: Can the records support a rate and interval for harm to legitimate users after a control change?

E5. Agent benefit: When all have the same information, does the agent with human review match or improve on the confirmed findings per reviewer hour of well-equipped human investigators, without materially more errors than either them or suitable decision rules that use no agent?

E6. Safeguards: How well do the agent's safeguards, covering prompt injection, approvals, its memory and the data it sends out, hold up against attackers who adapt to them? How much do they reduce the number of investigations completed?

E7. Learning: How quickly do useful lessons become available for reuse? How do testing against counterexamples, scoped promotion and withdrawal affect negative transfer (a reused lesson degrading later investigations)? How do they affect poisoning (attackers deliberately causing false lessons)?

E8. Release gate: Does the release gate catch planted wrong drafts under realistic volume and error prevalence, when reviewers and releasers do not know which drafts are planted, and how much reviewer time does it require? Some planted drafts rest a material claim only on a released item or on records linked to one, which tests C3's rule that such support adds no independent corroboration. Its criteria set no limit on correct drafts that the gate returns or lets expire. Passing E8 would show that the gate catches errors, not that it releases correct work at an acceptable rate; E5 bears on that cost.

E9. Channel states: Do C1's channel states reduce the absolute error of evaluation results against a mature holdout reference, compared with the platform's current label rule? The target is the selected metric on matured reported outcomes after a sampled review of report truth, not underlying fraud that may never be reported. Because the share of items that C1 reclassifies follows partly from its definitions and from the cohort's composition, E9 reports that share but does not count it as support; support requires a reduction in reference error exceeding the prespecified negligible improvement, with uncertainty accounted for.

Supplement S8 separately proposes practitioner review of whether the signals and responsibilities are available, useful and assigned to clear owners.

Supplement S7 gives the conditions and motivating sources for E1 to E7 and E9, and S7.1 states the tests for all nine. Four rules in S7.1 apply to every evaluation: an as-of cutoff on the information each method may use, outcomes decided blind under rules set in advance, a stated treatment of labels shaped by control decisions, and marking of test records. Of the S7.1 tests, E5 and E8 measure reviewer time, and E5 also compares cost across arms that see the same records. None

measures the storage, query work or controlled cross-account access that the added records require. S7 adds collection cost for E1's broader timing study and the safeguards' effect on investigations completed for E6. Passing the S7.1 tests would therefore not show that the added records justify their collection cost. E1, E5 and E8 also measure the work of identifiable staff on real case records, and E8's undisclosed planted drafts require a deception protocol. Each needs ethics and privacy review before it starts.

9.2 What the evidence can support

The specification has not been implemented or evaluated on platform data. Its traces demonstrate reasoning over hypothetical records, while safeguards, costs and investigator workload remain unmeasured. The practitioner protocol would assess availability and usefulness of the proposed records; it has not been run. C1's channel fields were derived from, and checked against, the illustrative provider's descriptions of card-network rules (Section 7.1). The networks' own rules were not read; the one network document cited is Visa's overview of its acquirer monitoring program (Section 8.2). Bank debits were compared only with two of the provider's pages and one bank's return-code table (Section 7.1); of the bank-debit candidates in Supplement S2, PAY-07 was judged covered and MM-03 was not reviewed. Instant push payments and stablecoin payments, which are also in the product scope, were not reviewed rail by rail; their mechanisms MM-07 and MM-12 were reviewed, but not their reporting channels (Supplement S2).

The product scope comes from one provider's public product catalog, roadmap and documentation, including roadmap items in public preview. Other platforms may combine products differently or face other mechanisms. Public interfaces expose only part of a provider's records, so "none identified" in Table 2 refers only to the pinned schemas and the terms searched. Collection ownership, retention and budgets remain deployment decisions.

The focused search and author-assigned mappings do not establish exhaustive coverage or the absence of equivalent systems. Bank case-management systems, card-network risk systems, vendor architectures and non-English literature were not reviewed as complete families. A second search on 28 and 29 October 2026 started from the paper's problem statement and the three requirement statements, without the related-work section, and logged 388 searches and page reads across patents, payment and fraud vendors, agent-governance and security-investigation products, standards and regulators, code repositories, journals and recent preprints. It found no source stating the three requirements together; the two sources it added are discussed in Section 2.3. The illustrative provider's public hiring priorities influenced selection and set most of the deeper reviews (Section 3; Supplement S2). A crosswalk to MITRE's Fight Fraud Framework finds that 18 of its 92 in-scope techniques have no retained mechanism; fourteen of them are covered only by candidates outside the signal map (Supplement S2.2) [102]. At the volume assumed in Section 6.4, the confirmatory release-gate test could run for more than a year and a half before it reaches its planned size (Supplement S7.1). The comparison supports the specified distinctions, not priority for their individual components.

Evidence strength varies by premise. Dhama's payment models are theoretical preprints [9, 10]. The general selective-label problem has broader support [1, 2], and the specification does not depend on those preprints' estimators or bounds being correct. Estimating a decision's effect still requires identifying assumptions [4]. Enforcement complaints motivate alleged mechanisms; vendor accounts support attributed observations within their stated settings.

Visibility also depends on where an agent runs. SIG-34, SIG-35, SIG-39 and SIG-40 require access inside the agent, its tool server or its outbound-traffic controls. A platform can obtain that for

its own agent but normally sees only interface activity from buyer- or merchant-operated agents. These signals cannot be assumed available in every deployment.

10. Conclusion

An investigation across payment products needs more than joined records. It needs to preserve why an outcome could be observed, what information was available to a query, and whether apparent support comes from an earlier conclusion.

This paper combines those requirements in one specification grounded in public evidence. The specification answers the three research questions. For RQ1, records keep their meaning across products through the seven record classes, the store contract and C2's coverage and learned-time records, which bound what an investigation could have established. Each analytical capability carries the limit on what its results can show. For RQ2, the agent tests explanations without operational authority. It works through governed queries whose coverage it cannot alter, and its drafts pass a checker, independent quality review and the human release of C3. Release authorizes reuse of a conclusion from organizational memory, while the conclusion keeps its status as context and its dependence on earlier agent outputs. For RQ3, feedback informs later work without concealing control effects or recycling conclusions. It rests on the decision-linked outcome channels of C1, which keep missing reports from silently becoming negatives, and on lessons that are tested, reviewed and released for a defined scope before reuse, monitored during it and, when they exonerate, re-tested at expiry. Decisions about accounts, payments and controls stay with authorized humans or accountable operational systems.

The 71 signals, 24 responsibilities and supporting interfaces make these choices inspectable. The hypothetical traces show their consequences: incomplete history leaves onset unresolved, a link learned late cannot enter a decision frozen before it was learned, a blocked payout closes a channel without producing a negative label, and transaction attribution changes which accounts bear reported outcomes. A stored assessment adds no independent observation.

For a platform that already logs decisions, queries and disputes, the specification states what those records and their joins must preserve. For designers and auditors of investigation systems, it places an agent's access, its evidential claims and its reuse of conclusions in one inspectable design.

The next test is operational value. Evaluations must determine whether these records improve conclusions, whether the release gate works under realistic demand, and whether the agent saves more work than review consumes. The specification supplies a concrete basis for those tests without presuming their result.

Author and publication declarations

Acknowledgments. Thanks to Jiahong Dai (Sections 2.2 and 6.2), Brian Freeman (Section 2.3), Oleg Solozobov (Sections 2.1, 2.4, 5.2, 5.3 and 7.1) and Xuwei Tan (Sections 2.4 and 9.1) for feedback.

Use of generative AI. The author used Anthropic Claude and OpenAI ChatGPT/Codex in the research, as Section 3 describes, and for drafting and revision. The author takes responsibility for the content.

Data and code availability. The accompanying supplement provides the signal inventory, mappings, entity and interface records, source locators, schema pins and synthetic worked-example data. Reproduction scripts check the included counts, lineage and arithmetic and rebuild the

documents. No private platform data, implementation of the proposed architecture or empirical system-evaluation code is supplied. External source texts remain with their publishers or authors. These files are published with this preprint (https://doi.org/10.5281/zenodo.23772005), and the working draft and review record are at https://github.com/matkowsky/payment-fraud.

Funding. This work received no funding.

License. Creative Commons Attribution 4.0 International (CC BY 4.0) applies to the author's text, figures and supplement tables. The Stripe schema-derived components of two supplement tables remain under the MIT License, as THIRD_PARTY_NOTICES.md states.

Ethics and confidentiality. No human participants were involved. The paper uses only public sources, and no confidential information from any current or former employer was used. The practitioner review protocol in Supplement S8 has not been run and would need the ethics review that the involved institutions require.

Competing interests. The author previously worked for Microsoft and is seeking employment in security, fraud prevention, and trust and safety. The author has no current employment or consulting relationship. No company has received a draft of this work. The STIX 2.1 Errata 01 acknowledgments list the author, then at Microsoft, among participants of the OASIS Cyber Threat Intelligence Technical Committee; the paper cites the 2021 STIX 2.1 standard.

Release status. This is preprint version 1.0, dated 29 October 2026. It has not been peer reviewed.

References

[1] H. Lakkaraju, J. Kleinberg, J. Leskovec, J. Ludwig, and S. Mullainathan. 2017. The Selective Labels Problem: Evaluating Algorithmic Predictions in the Presence of Unobservables. *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 275–284. https://doi.org/10.1145/3097983.3098066.

[2] N. Kilbertus, M. Gomez-Rodriguez, B. Schölkopf, K. Muandet, and I. Valera. 2020. Fair Decisions Despite Imperfect Predictions. *Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics*, 108, pp. 277–287. https://proceedings.mlr.press/v108/kilbertus20a.html.

[3] L. Bottou, J. Peters, J. Quiñonero-Candela, D. X. Charles, D. M. Chickering, E. Portugaly, D. Ray, P. Simard, and E. Snelson. 2013. Counterfactual Reasoning and Learning Systems: The Example of Computational Advertising. *Journal of Machine Learning Research*, 14(101), pp. 3207–3260. https://jmlr.org/papers/v14/bottou13a.html.

[4] M. A. Hernán and J. M. Robins. 2020. Causal Inference: What If. Boca Raton, Chapman & Hall/CRC. Chapters cited: 1, 2, 3, 7 and 19. Cited version: author-hosted revision (19 August 2026). https://miguelhernan.org/whatifbook.

[5] A. Ehrhardt, C. Biernacki, V. Vandewalle, P. Heinrich, and S. Beben. 2021. Reject Inference Methods in Credit Scoring. *Journal of Applied Statistics*, 48(13-15), pp. 2734–2754. https://doi.org/10.1080/02664763.2021.1929090.

[6] S. Cao, X. Yang, C. Chen, J. Zhou, X. Li, and Y. Qi. 2019. TitAnt: Online Real-time Transaction Fraud Detection in Ant Financial. *Proceedings of the VLDB Endowment*, 12(12), pp. 2082–2093. https://doi.org/10.14778/3352063.3352126.

[7] A. Dal Pozzolo, G. Boracchi, O. Caelen, C. Alippi, and G. Bontempi. 2018. Credit Card Fraud Detection: A Realistic Modeling and a Novel Learning Strategy. *IEEE Transactions on Neural Networks and Learning Systems*, 29(8), pp. 3784–3797. https://doi.org/10.1109/TNNLS.2017.2736643.

[8] N. Tax, K. J. de Vries, M. de Jong, N. Dosoula, B. van den Akker, J. Smith, O. Thuong, and L. Bernardi. 2021. Machine Learning for Fraud Detection in E-Commerce: A Research Agenda. arXiv preprint 2107.01979v1. https://arxiv.org/abs/2107.01979v1.

[9] G. Dhama. 2026. Causal Label Recovery in Payment Networks. arXiv preprint 2605.29272v1. https://arxiv.org/abs/2605.29272v1.

[10] G. Dhama. 2026. The Fundamental Limits of Fraud Detection in Card Payment Networks: An Information-Theoretic Theory of Delayed, Censored, Corrupted, and Counterfactual Feedback. arXiv preprint 2605.27557v2. https://arxiv.org/abs/2605.27557v2.

[11] M. He and J. L. Loh. 2026. Combating Organized Platform Abuse: Amplifying Weak Risk Signals with Structural Information. arXiv preprint 2605.07383v2. https://arxiv.org/abs/2605.07383v2.

[12] O. Chapelle. 2014. Modeling Delayed Feedback in Display Advertising. *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 1097–1105. https://doi.org/10.1145/2623330.2623634.

[13] M. Manapat. 2015. Counterfactual evaluation of machine learning models. Conference talk, PyData Seattle. https://pydata.org/seattle2015/schedule/presentation/17/.

[14] A. Egg. 2025. Off-policy Evaluation for Payments at Adyen. arXiv preprint 2501.10470v1. https://arxiv.org/abs/2501.10470v1.

[15] O. Solozobov. 2026. Decision Trace Schema for Governance Evidence in Real-Time Risk Systems. arXiv preprint 2604.09296v1. https://arxiv.org/abs/2604.09296v1.

[16] M. Wu, L. Wang, Y. Zhang, and J. Deng. 2026. Evidence Coverage for Intent-Bound Execution: Scope, Obligations, and Cutoff Reasoning. arXiv preprint 2610.01386v1. https://arxiv.org/abs/2610.01386v1.

[17] R. Snodgrass and I. Ahn. 1986. Temporal Databases. *Computer*, 19(9), pp. 35–42. https://doi.org/10.1109/MC.1986.1663327.

[18] Z. Wang. 2026. TOKI: A Bitemporal Operator Algebra for Contradiction Resolution in LLM-Agent Persistent Memory. arXiv preprint 2606.06240v1. https://arxiv.org/abs/2606.06240v1.

[19] P. Rasmussen, P. Paliychuk, T. Beauvais, J. Ryan, and D. Chalef. 2025. Zep: A Temporal Knowledge Graph Architecture for Agent Memory. arXiv preprint 2501.13956v1. https://arxiv.org/abs/2501.13956v1.

[20] A. Dan. 2026. Anamnesis: An Authority-Preserving Runtime Contract for Persistent AI Memory. Zenodo preprint, Version 1.0, 16 August 2026. https://zenodo.org/records/22118090.

[21] S. Razniewski and W. Nutt. 2011. Completeness of Queries over Incomplete Databases. *Proceedings of the VLDB Endowment*, 4(11), pp. 749–760. https://doi.org/10.14778/3402707.3402715.

[22] S. Brown. 2026. Quipu: A Governed Bitemporal Knowledge Graph Store. arXiv preprint 2608.16813v1. https://arxiv.org/abs/2608.16813v1.

[23] K. Mukherjee and M. Kantarcioglu. 2025. LLM-driven Provenance Forensics for Threat Investigation and Detection. arXiv preprint 2508.21323v2. The v2 PDF title page reads "LLM-driven Provenance Forensics for Threat Intelligence and Detection" instead. https://arxiv.org/abs/2508.21323v2.

[24] World Wide Web Consortium. 2013. PROV-DM: The PROV data model. W3C Recommendation. https://www.w3.org/TR/2013/REC-prov-dm-20130430/.

[25] J. Singh, J. Cobbe, and C. Norval. 2019. Decision Provenance: Harnessing Data Flow for Accountable Systems. *IEEE Access*, 7, pp. 6562–6574. https://doi.org/10.1109/ACCESS.2018.2887201.

[26] OASIS. 2021. STIX Version 2.1. OASIS Standard. https://docs.oasis-open.org/cti/stix/v2.1/os/stix-v2.1-os.html.

[27] Office of the Director of National Intelligence. 2015. Intelligence Community Directive 203: Analytic Standards. Directive. As amended by technical amendments through 21 January 2022. https://archive.dni.gov/files/documents/ICD/ICD-203.pdf.

[28] J. Dai, Z. Yang, P. Shao, K. Ng, Z. Liu, C. Ju, Y. He, and B. Hu. 2026. Hearsay: Can an Auditor Trust the Record a Deployed Agent Harness Writes? arXiv preprint 2609.32495v1. https://arxiv.org/abs/2609.32495v1.

[29] J. Doyle. 1979. A Truth Maintenance System. *Artificial Intelligence*, 12(3), pp. 231–272. https://doi.org/10.1016/0004-3702(79)90008-0.

[30] B. Malheiro, N. R. Jennings, and E. Oliveira. 1994. Belief Revision in Multi-Agent Systems. *Proceedings of the 11th European Conference on Artificial Intelligence*, pp. 294–298. https://eprints.soton.ac.uk/252143.

[31] H. Hettema. 2021. Rationality Constraints in Cyber Defense: Incident Handling, Attribution and Cyber Threat Intelligence. *Computers & Security*, 109, Article 102396. https://doi.org/10.1016/j.cose.2021.102396.

[32] T. R. Sumers, S. Yao, K. Narasimhan, and T. L. Griffiths. 2024. Cognitive Architectures for Language Agents. *Transactions on Machine Learning Research*. https://arxiv.org/abs/2309.02427v3.

[33] Y. Wang, J. Zhang, Z. Wu, T. Cai, Z. Liu, Z. Sun, M. Dong, M. Zheng, Y. Duan, X. Yin, and Y. Zhu. 2026. From Agent Traces to Trust: A Survey of Evidence Tracing and Execution Provenance in LLM Agents. arXiv preprint 2606.04990v5. https://arxiv.org/abs/2606.04990v5.

[34] Q. Zhu, S. Chen, R. Yu, Z. Wu, and B. Wang. 2026. From Lossy to Verified: A Provenance-Aware Tiered Memory for Agents. arXiv preprint 2602.17913v1. Also presented as a poster at the ICLR 2026 Workshop on Memory for LLM-Based Agentic Systems (MemAgents). Cited version: arXiv 2602.17913v1 (20 February 2026). https://arxiv.org/abs/2602.17913v1.

[35] Y. Qi, X. Xu, and Y. Li. 2026. When Not to Write Memory: Governing False Promotion from Correlated Agent Traces. arXiv preprint 2607.02579v1. https://arxiv.org/abs/2607.02579v1.

[36] H. Akhtar and B. Freeman. 2026. Contestable Agents for Data Governance: Recording Human Stewardship of AI-Generated Metadata Across Structured and Unstructured Systems. arXiv preprint 2610.31553v1. https://arxiv.org/abs/2610.31553v1.

[37] S. Mitra, R. Patel, S. Mittal, M. R. Rahman, and S. Rahimi. 2026. AgenticCyOps: Securing Multi-Agentic AI Integration in Enterprise Cyber Operations. arXiv preprint 2603.09134v1. https://arxiv.org/abs/2603.09134v1.

[38] M. Albarrak and S. Jagtap. 2026. Designing an AI-Assisted Cyber Threat Intelligence Framework for Industry 4.0: A Human-in-the-Loop Design Science Approach. *Applied Sciences*, 16(15), Article 7646. https://doi.org/10.3390/app16157646.

[39] R. Drapeau and S. Samant. 2026. Scaling Radar: Fighting fraud with agent-assisted machine learning. Stripe developer blog. https://stripe.dev/blog/scaling-radar-fighting-fraud-with-agent-assisted-machine-learning. Accessed 26 October 2026.

[40] A. Alten. 2026. When Does Persistent Agent Memory Become Accountable? Scalytics blog. https://www.scalytics.io/blog/agent-memory-governance-decision-records. Accessed 26 October 2026.

[41] K. Talamadupula. 2026. Oracle AI Agent Memory Brings Persistent Enterprise Context to NVIDIA OpenShell. Oracle Database blog. https://blogs.oracle.com/database/oracle-ai-agent-memory-brings-persistent-enterprise-context-to-nvidia-openshell. Accessed 26 October 2026.

[42] C. Koch. 2026. From Agent Output to Authorized Transition. arXiv preprint 2609.28216v1. The v1 PDF title page reads "From Agent Output to Authorized Transition: The Agile-V Assurance Spine for State-Bound Evidence in Software, Firmware, and Hardware Engineering" instead. https://arxiv.org/abs/2609.28216v1.

[43] M. Bara. 2026. Epistemic Sybil Resistance: Multiplying AI Agents Without Multiplying Evidence. arXiv preprint 2609.01873v1. https://arxiv.org/abs/2609.01873v1.

[44] G. Evron. 2026. Adversary Brains: Hypothesis-Based Logical Signatures for Threat Actor Detection with LLMs. arXiv preprint 2610.35624v1. https://arxiv.org/abs/2610.35624v1.

[45] Z. Liu, C. Chen, X. Yang, J. Zhou, X. Li, and L. Song. 2018. Heterogeneous Graph Neural Networks for Malicious Account Detection. *Proceedings of the 27th ACM International Conference on Information and Knowledge Management*. https://doi.org/10.1145/3269206.3272010.

[46] Stripe. 2026. Query disputes and fraud data. Stripe documentation. https://docs.stripe.com/data/query-disputes-and-fraud-data. Accessed 26 October 2026.

[47] Stripe. 2026. Disputes analytics. Stripe documentation. https://docs.stripe.com/payments/analytics/disputes. Accessed 26 October 2026.

[48] Stripe. 2026. The Scheduled Query object. Stripe documentation. https://docs.stripe.com/api/sigma/scheduled_queries/object. Accessed 26 October 2026.

[49] Adyen. 2026. Monitor risk performance. Adyen documentation. https://docs.adyen.com/risk-management/dispute-and-fraud-monitoring. Accessed 26 October 2026.

[50] C. Erdbrink and L. Ellis. 2023. Analyzing a Transaction in a Payment Processing System. US patent application publication, US20230298028A1, Fidelity Information Services, LLC. https://patents.google.com/patent/US20230298028A1/en.

[51] G. Sakellariou, P. Fouliras, I. Mavridis, and P. Sarigiannidis. 2022. A Reference Model for Cyber Threat Intelligence (CTI) Systems. *Electronics*, 11(9), Article 1401. https://doi.org/10.3390/electronics11091401.

[52] O. Solozobov. 2026. Governed Auditable Decisioning Under Uncertainty: Synthesis and Agentic Extension. arXiv preprint 2604.19112v2. Cited version: arXiv 2604.19112v2 (2026). https://arxiv.org/abs/2604.19112v2.

[53] X. Tan, Y. Ma, and X. Zhang. 2026. SR-Fraud: An Outcome-Supervised Reflective LLM Agent Framework for Non-Stationary Payment Fraud Detection. arXiv preprint 2609.27287v1. https://arxiv.org/abs/2609.27287v1.

[54] R. Sharma. 2026. Toward Auditable Fraud Detection: Combining Graph Features, Model Explanations, and Agentic Case Investigation. arXiv preprint 2607.19266v1. https://arxiv.org/abs/2607.19266v1.

[55] Stripe. 2026. ARG Engineering Manager. Stripe Careers. https://stripe.com/careers/listing/arg-engineering-manager/8113337. Accessed 21 October 2026.

[56] Stripe. 2026. Abuse Research Engineer. Stripe Careers. https://stripe.com/careers/listing/abuse-research-engineer/8172503. Accessed 21 October 2026.

[57] Stripe. 2026. OpenAPI specification for the Stripe API. GitHub, stripe/openapi. https://github.com/stripe/openapi/tree/18fa2cc768024b47789dc6fa243acd48478acac2. Accessed 22 October 2026.

[58] M. Meli, M. R. McNiece, and B. Reaves. 2019. How Bad Can It Git? Characterizing Secret Leakage in Public GitHub Repositories. *Network and Distributed Systems Security (NDSS) Symposium 2019*. https://doi.org/10.14722/ndss.2019.23418.

[59] N. Demir, Y. Vekaria, G. Smaragdakis, and Z. Durumeric. 2026. Keys on Doormats: Exposed API Credentials on the Web. arXiv preprint 2603.12498v3. Accepted at ACM CCS 2026 (November 2026) per the reference printed in arXiv v3. https://arxiv.org/abs/2603.12498v3.

[60] W. Gamazo and N. Quist. 2023. CloudKeys in the Air: Tracking Malicious Operations of Exposed IAM Keys. Palo Alto Networks Unit 42. https://unit42.paloaltonetworks.com/malicious-operations-of-exposed-iam-keys-cryptojacking/. Accessed 19 October 2026.

[61] Microsoft Threat Intelligence. 2022. From cookie theft to BEC: Attackers use AiTM phishing sites as entry point to further financial fraud. Microsoft Security Blog. https://www.microsoft.com/en-us/security/blog/2022/07/12/from-cookie-theft-to-bec-attackers-use-aitm-phishing-sites-as-entry-point-to-further-financial-fraud/. Accessed 19 October 2026.

[62] P. Doerfler, M. Marincenko, J. Ranieri, Y. Jiang, A. Moscicki, D. McCoy, and K. Thomas. 2019. Evaluating Login Challenges as a Defense Against Account Takeover. *Proceedings of the 2019 World Wide Web Conference*, pp. 372–382. https://doi.org/10.1145/3308558.3313481.

[63] Microsoft Threat Intelligence. 2023. Threat actors misuse OAuth applications to automate financially driven attacks. Microsoft Security Blog. https://www.microsoft.com/en-us/security/blog/2023/12/12/threat-actors-misuse-oauth-applications-to-automate-financially-driven-attacks/. Accessed 19 October 2026.

[64] M. Campobasso and L. Allodi. 2020. Impersonation-as-a-Service: Characterizing the Emerging Criminal Infrastructure for User Impersonation at Scale. *Proceedings of the 2020 ACM SIGSAC*

Conference on Computer and Communications Security, pp. 1665–1680. https://doi.org/10.1145/3372297.3417892.

[65] Federal Bureau of Investigation, Internet Crime Complaint Center. 2018. Cybercriminals Utilize Social Engineering Techniques to Obtain Employee Credentials to Conduct Payroll Diversion. FBI IC3. https://www.ic3.gov/PSA/2018/PSA180918. Accessed 19 October 2026.

[66] Financial Crimes Enforcement Network. 2014. Advisory FIN-2014-A005: Update on U.S. Currency Restrictions in Mexico: Funnel Accounts and TBML. FinCEN. https://www.fincen.gov/system/files/advisory/FIN-2014-A005.pdf. Accessed 19 October 2026.

[67] M. Bitaab, A. Karimi, Z. Lyu, A. Oest, D. Kuchhal, M. Saad, G.-J. Ahn, R. Wang, T. Bao, Y. Shoshitaishvili, and A. Doupé. 2025. ScamMagnifier: Piercing the Veil of Fraudulent Shopping Website Campaigns. *Network and Distributed System Security (NDSS) Symposium 2025*. https://doi.org/10.14722/ndss.2025.240763.

[68] D. McCoy, H. Dharmdasani, C. Kreibich, G. M. Voelker, and S. Savage. 2012. Priceless: The Role of Payments in Abuse-advertised Goods. *Proceedings of the 2012 ACM Conference on Computer and Communications Security*, pp. 845–856. https://doi.org/10.1145/2382196.2382285.

[69] H. Tian, S. M. Gaffigan, D. S. West, and D. McCoy. 2018. Bullet-Proof Payment Processors. *2018 APWG Symposium on Electronic Crime Research*, pp. 1–11. https://doi.org/10.1109/eCrime.2018.8376208.

[70] Federal Trade Commission. 2020. FTC v. First Data Merchant Services LLC, No. 1:20-cv-03867 (S.D.N.Y.), Complaint, ECF 1. US District Court, Southern District of New York. Filed complaint; related stipulated order and nonadmission terms identified in S1. https://www.ftc.gov/system/files/documents/cases/first_data_filed_complaint.pdf. Accessed 22 October 2026.

[71] Federal Trade Commission. 2025. FTC v. Paddle.com Market Limited, No. 1:25-cv-01886 (D.D.C.), Complaint, ECF 1. US District Court, District of Columbia. Filed complaint; related stipulated order and nonadmission terms identified in S1. https://www.ftc.gov/system/files/ftc_gov/pdf/PaddleComplaintForPermanentInjunction,MonetaryJudgment,andOtherRelief.pdf. Accessed 21 October 2026.

[72] Federal Trade Commission. 2019. FTC v. AlliedWallet, Inc., No. 2:19-cv-04355 (C.D. Cal.), Complaint, ECF 24. US District Court, Central District of California. Filed complaint; related stipulated order and nonadmission terms identified in S1. https://www.ftc.gov/system/files/documents/cases/alliedwallet_complaint_7-3-19.pdf. Accessed 21 October 2026.

[73] Stripe. 2026. How disputes work. Stripe documentation. https://docs.stripe.com/disputes/how-disputes-work. Accessed 25 October 2026.

[74] S. Zhang, E. He, X. Zhan, S. He, R. Xiao, X. Yi, and H. Li. 2026. Generative AI-Enabled Refund Fraud in Chinese E-Commerce: Investigation on Merchants and Platform Workers. arXiv preprint 2606.03215v1. https://arxiv.org/abs/2606.03215v1.

[75] Stripe. 2026. Customer account evaluations. Stripe documentation. https://docs.stripe.com/radar/customer-account-evaluations. Accessed 19 October 2026.

[76] Stripe. 2026. Free trial abuse prevention. Stripe documentation. https://docs.stripe.com/radar/free-trial-abuse. Accessed 19 October 2026.

[77] Stripe. 2026. Pay-as-you-go abuse evaluation. Stripe documentation. https://docs.stripe.com/radar/pay-as-you-go-abuse. Accessed 19 October 2026.

[78] Stripe. 2026. Account farming prevention. Stripe documentation. Preview feature; Radar Pro plan required. https://docs.stripe.com/radar/account-farming. Accessed 19 October 2026.

[79] Stripe. 2026. Dispute and fraud card monitoring programs. Stripe documentation. https://docs.stripe.com/disputes/monitoring-programs. Accessed 25 October 2026.

[80] Stripe. 2026. ACH Payment Terms. Stripe Legal. https://stripe.com/legal/ACH. Accessed 25 October 2026.

[81] K. Thomas, D. McCoy, C. Grier, A. Kolcz, and V. Paxson. 2013. Trafficking Fraudulent Accounts: The Role of the Underground Market in Twitter Spam and Abuse. *Proceedings of the 22nd USENIX Security Symposium*, pp. 195–210. https://www.usenix.org/conference/usenixsecurity13/technical-sessions/paper/thomas.

[82] Z. Chen, D. Luo, Y. Hu, L. Wu, B. He, and Y. Zhou. 2025. Dissecting Payload-based Transaction Phishing on Ethereum. *Network and Distributed System Security (NDSS) Symposium 2025*. https://doi.org/10.14722/ndss.2025.230311.

[83] Stripe. 2026. Onramp API reference. Stripe documentation. Public preview. https://docs.stripe.com/crypto/using-the-api. Accessed 19 October 2026.

[84] R. Parasuraman and D. H. Manzey. 2010. Complacency and Bias in Human Use of Automation: An Attentional Integration. *Human Factors*, 52(3), pp. 381–410. https://doi.org/10.1177/0018720810376055.

[85] J. M. Wolfe, T. S. Horowitz, and N. M. Kenner. 2005. Rare Items Often Missed in Visual Searches. *Nature*, 435, pp. 439–440. https://doi.org/10.1038/435439a.

[86] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz. 2023. Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, pp. 79–90. Cited version: arXiv 2302.12173v2 (5 May 2023). https://doi.org/10.1145/3605764.3623985.

[87] E. Debenedetti, J. Zhang, M. Balunovic, L. Beurer-Kellner, M. Fischer, and F. Tramèr. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. *Advances in Neural Information Processing Systems 37, Datasets and Benchmarks Track*, pp. 82895–82920. https://doi.org/10.52202/079017-2636.

[88] A. A. Kumar. 2026. Loopjacking: Hijacking Human-in-the-Loop Approval. arXiv preprint 2609.21081v1. https://arxiv.org/abs/2609.21081v1.

[89] L. Beurer-Kellner and M. Fischer. 2025. MCP Security Notification: Tool Poisoning Attacks. Invariant Labs. https://invariantlabs.ai/blog/mcp-security-notification-tool-poisoning-attacks. Accessed 19 October 2026.

[90] P. Dash, T. Ge, A. Jain, T. Shah, and Z. Shang. 2026. From Untrusted Input to Trusted Memory: A Systematic Study of Memory Poisoning Attacks in LLM Agents. arXiv preprint 2606.04329v2. https://arxiv.org/abs/2606.04329v2.

[91] M. Nasr, N. Carlini, C. Sitawarin, S. V. Schulhoff, J. Hayes, M. Ilie, J. Pluto, S. Song, H. Chaudhari, I. Shumailov, A. Thakurta, K. Y. Xiao, A. Terzis, and F. Tramèr. 2025. The

Attacker Moves Second: Stronger Adaptive Attacks Bypass Defenses Against LLM Jailbreaks and Prompt Injections. arXiv preprint 2510.09023v1. https://arxiv.org/abs/2510.09023v1.

[92] Stripe. 2026. Authenticate with 3D Secure. Stripe documentation. https://docs.stripe.com/payments/3d-secure/authentication-flow. Accessed 19 October 2026.

[93] Stripe. 2026. SEPA Direct Debit payments. Stripe documentation. https://docs.stripe.com/payments/sepa-debit. Accessed 25 October 2026.

[94] Lead Bank. 2026. Returns, Reversals, & Exceptions. Lead Bank documentation. https://docs.lead.bank/api-integrations/ach/return-codes. Accessed 25 October 2026.

[95] K. Fatima, M. Schuckers, G. Cruz-Ortiz, D. Hou, S. Purnapatra, T. Andrews, A. Neupane, B. Marshall, and S. Schuckers. 2024. A Large-Scale Study of Performance and Equity of Commercial Remote Identity Verification Technologies Across Demographics. *2024 IEEE International Joint Conference on Biometrics*, pp. 1–8. https://doi.org/10.1109/IJCB62174.2024.10744432.

[96] J. Meltzer and V. Chadalapaka. 2025. How Stripe Radar responded to a new wave of card testing. Stripe blog. https://stripe.com/blog/how-stripe-radar-responded-to-a-new-wave-of-card-testing. Accessed 21 October 2026.

[97] A. Dimitriadis, A. Papoutsis, D. Kavalieros, T. Tsikrika, S. Vrochidis, and I. Kompatsiaris. 2025. EVACTI: Evaluating the Actionability of Cyber Threat Intelligence. *International Journal of Information Security*, 24, Article 123. https://doi.org/10.1007/s10207-025-01033-z.

[98] Z. Xiong, Y. Lin, W. Xie, P. He, Z. Liu, J. Tang, H. Lakkaraju, and Z. Xiang. 2026. How Memory Management Impacts LLM Agents: An Empirical Study of Experience-Following Behavior. *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 623–645. https://doi.org/10.18653/v1/2026.acl-long.27.

[99] Stripe. 2026. Best practices for risk management. Stripe documentation. https://docs.stripe.com/connect/risk-management/best-practices. Accessed 19 October 2026.

[100] Stripe. 2026. Understand how charges work in a Connect integration. Stripe documentation. https://docs.stripe.com/connect/charges. Accessed 19 October 2026.

[101] Visa. 2025. Visa Acquirer Monitoring Program Overview. Visa. https://corporate.visa.com/content/dam/VCOM/corporate/visa-perspectives/security-and-trust/documents/visa-acquirer-monitoring-program-fact-sheet-2025.pdf. Accessed 25 October 2026.

[102] MITRE Center for Threat-Informed Defense. 2026. Fight Fraud Framework (F3), version 1.1. GitHub, center-for-threat-informed-defense/fight-fraud-framework. Cited version: commit cb186c2 (18 September 2026). https://github.com/center-for-threat-informed-defense/fight-fraud-framework/tree/cb186c2b3372164e0b7def0d4f09531b6acc16e5. Accessed 25 October 2026.